



**Politecnico
di Torino**

Exercise 1: 5 D.O.F. robot on Adams

Master's Degree in Automotive Engineering

Maurizio Pio Vergara: `s346643@studenti.polito.it`

Loris Fonseca: `s342696@studenti.polito.it`

Renis Shtylla: `s344685@studenti.polito.it`

a.a 2024/2025

Contents

1	Objective	3
2	Methodology	3
3	Trajectory	3
4	Results	5
5	Conclusions	6

1 Objective

The objective of this work is to design and implement a 5-axis robot in Adams, featuring an articulated arm with 3 degrees of freedom and a wrist with 2 degrees of freedom. Adams is a multibody dynamics and motion analysis software that enables the study of load and force distribution within mechanical systems. The software incorporates real-world physics by solving equations for kinematics, statics, quasi-statics, and dynamics simultaneously.

2 Methodology

The first step is to implement the rigid bodies that compose the robot. Each part of the robot is modeled using box-shaped bodies. The base, designated as body 0, is fixed. Local reference frames have been defined for each body at their appropriate positions, as shown in Figure 1.

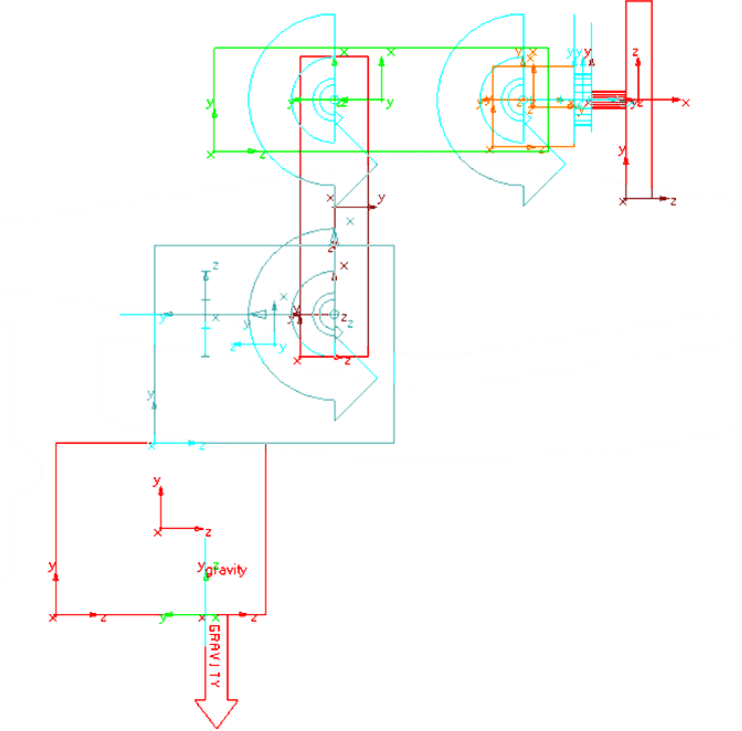


Figure 1: 5 D.O.F. robot - Reference frames

Rotary joints have been added using these local reference frames. The reference frames are configured such that the z-axis serves as the axis of rotation for each body. For every rotary joint, a motion has been implemented using the AKISPL function. These motions are of the velocity type, so an initial starting position was required for each joint. These initial positions were provided in the file "Trajectory_01_initial_conditions.txt".

The velocity characteristics for each joint were specified for all time intervals, and this data was integrated into the model using a 2D spline applied within the AKISPL function mentioned earlier.

3 Trajectory

Using the implemented motions and initial conditions mentioned in the previous section, the trajectories of the robot's end effector and wrist center point were obtained. The velocity profiles, defined as functions of time through the AKISPL implementation, enabled accurate simulation of the robot's motion.

By leveraging the capabilities of Adams software, the positions and orientations of both the end effector and the wrist center point were computed throughout the robot's operation cycle. These trajectories are displayed in Figures 2 and 3.

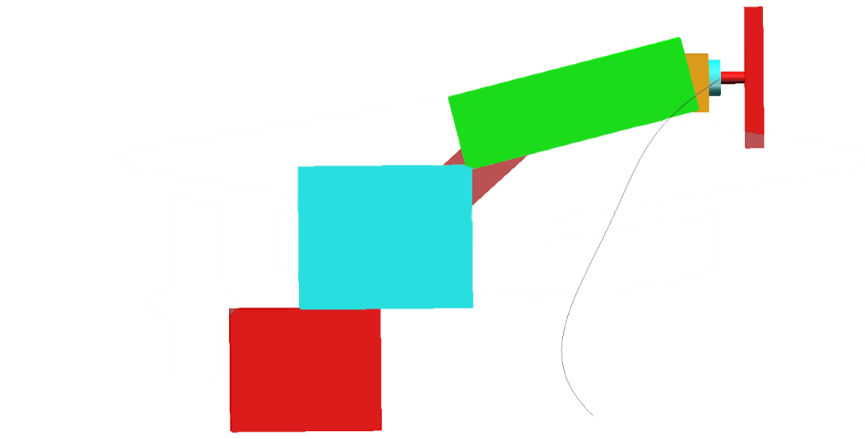


Figure 2: End effector trajectory

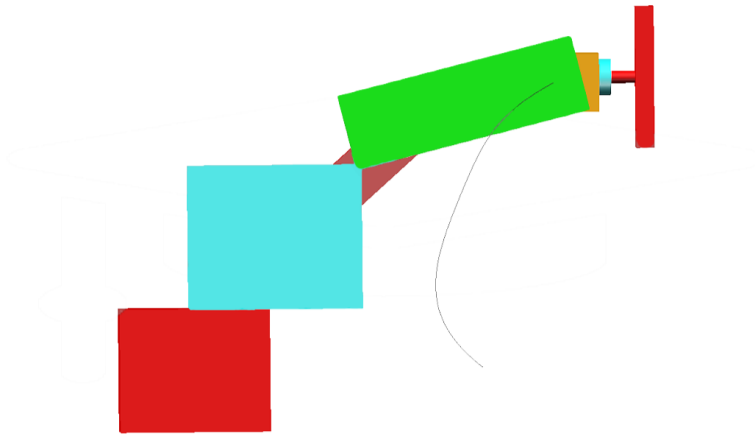


Figure 3: Wrist center point trajectory

By starting and completing the simulation, it is also easy to obtain both the initial and final positions of the robot, as shown in Figure 4 and 5.

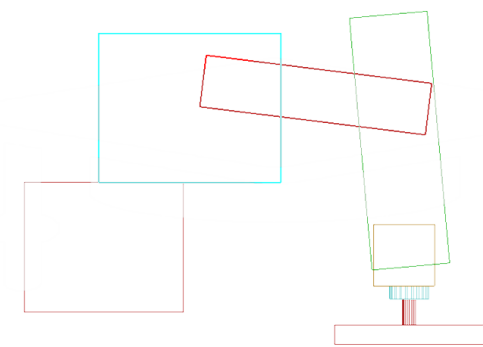


Figure 4: Robot initial position

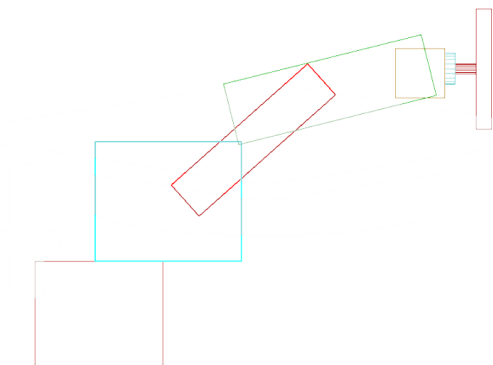


Figure 5: Robot final position

4 Results

Finally, the software allows for verification of the kinematics and dynamics results, based on the previously mentioned conditions, as well as the masses of the joints and the payload attached to the end effector. The following figures show the angular velocity ω of each joint about its center of mass with respect to the global coordinate system.

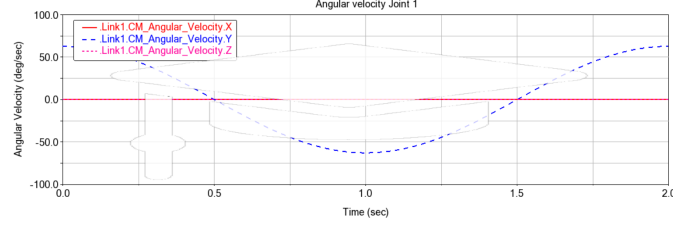


Figure 6: Angular velocity of joint 1

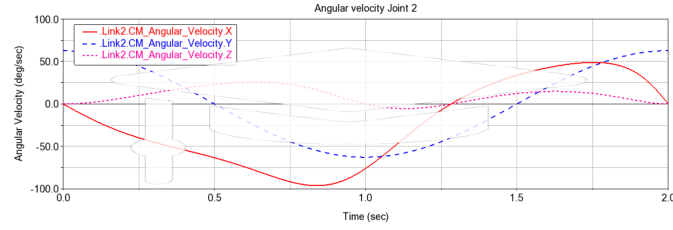


Figure 7: Angular velocity of joint 2

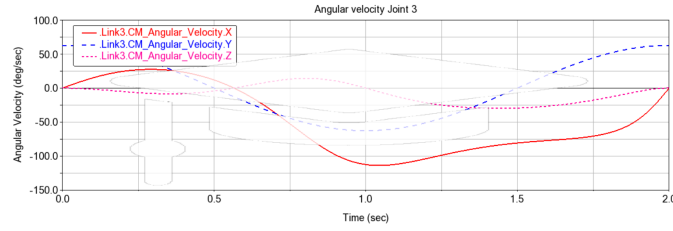


Figure 8: Angular velocity of joint 3

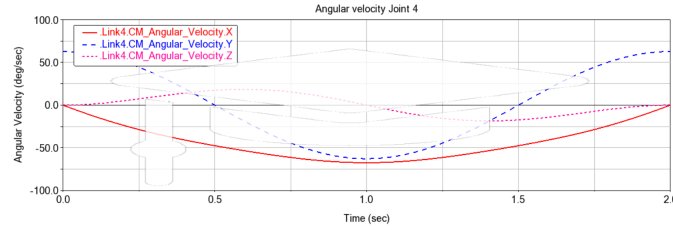


Figure 9: Angular velocity of joint 4

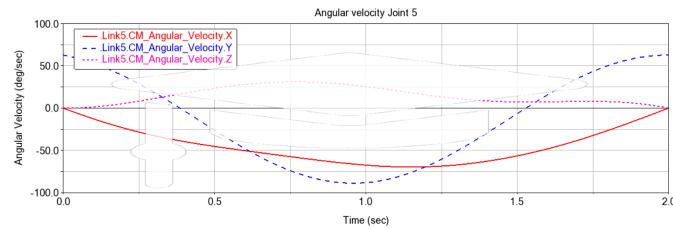


Figure 10: Angular velocity of joint 5

5 Conclusions

In this report, the design and implementation of a 5-degree-of-freedom robot using Adams software were successfully conducted. The articulated robotic arm, with three degrees of freedom in the arm and two in the wrist, showcased the following key outcomes:

- **Design Implementation:** the robot was effectively modeled using rigid box-shaped bodies, with each component linked via rotary joints. The AKISPL function facilitated precise motion control by defining velocity profiles for the joints.
- **Trajectory Analysis:** the trajectories of the robot's end effector and wrist center point were accurately simulated, demonstrating the software's capability to compute their positions and orientations through time. Initial and final positions validated the expected motion paths.
- **Kinematic Results:** the analysis yielded valuable insights into the angular velocities of each joint, ensuring the system's behavior adhered to the specified motion parameters.

Overall, this work highlights the robustness of Adams software in simulating and analyzing the mechanical dynamics of robotic systems. The methodologies applied provide a reliable framework for kinematic evaluation of each joint.



**Politecnico
di Torino**

Exercise 2: Rotation matrix multiplication

Master's Degree in Automotive Engineering

Maurizio Pio Vergara: `s346643@studenti.polito.it`

Loris Fonseca: `s342696@studenti.polito.it`

Renis Shtylla: `s344685@studenti.polito.it`

a.a 2024/2025

Contents

1	Objective	3
2	Methodology	3
3	Positioning matrix (${}^0\hat{\mathbf{A}}_1$) of the triangle ABC	3
4	Unit vectors of the reference frames and triangle ABC plot	3
5	Rotation of the initial triangle of 90° about local axis y_1	5
6	Rotation of the initial triangle of -90° about global axis y	6
7	Rotation of the last triangle of 90° about local axis x_2	8
8	Conclusions	9

1 Objective

The primary aim of the exercise is to implement a triangle in MATLAB based on its given coordinates relative to the fixed reference frame $Oxyz$, and the mobile reference frame $O1x1y1z1$, which is attached to the body and fixed at the midpoint of the triangle's base. The secondary aim is to plot three different rotations about the axes in either the fixed or mobile reference frame, according to the angles specified in the requirements.

2 Methodology

First, the homogeneous matrices of the fixed reference frame are loaded into MATLAB, along with the vectors defining the positions of the vertices A , B , C of the triangle and the origin of the mobile reference frame. The given data to be used for solving subsequent questions are reported in the following code:

```
1  clc
2  clear
3  close all
4
5  %% Unit vectors related to the fixed reference frame
6  scale=1.5;
7  i = [1 0 0 0]*scale;
8  j = [0 1 0 0]*scale;
9  k = [0 0 1 0]*scale;
10
11 %% Triangle in initial position
12 pA_o = [0 1 1 1]';          % given by the data
13 pB_o = [0 7 1 1]';          % the apix ' is used to have a 4x1 vectors
14 pC_o = [0 4 7 1]';
15 p01_o = [0 4 1 1]';
```

3 Positioning matrix (${}^0\hat{A}_1$) of the triangle ABC

The homogeneous orientation matrix ${}^0\hat{A}_1$ of the triangle with respect to system 0 has been determined graphically by analyzing the given information. The provided graphic allows for visualizing the direction of the vectors and the position of the origin of the mobile reference frame in its initial position. The following code demonstrates how the matrix ${}^0\hat{A}_1$ has been implemented on MATLAB.

```
1  %% A10 homogeneous matrix
2  A10o = [0 0 1 0; ...      % Implemented by using Exercise data
3          1 0 0 4; ...
4          0 1 0 1; ...
5          0 0 0 1];
6  A10 = [0 0 1; ...
7         1 0 0; ...
8         0 1 0];
```

4 Unit vectors of the reference frames and triangle ABC plot

Unit vectors associated with the fixed and mobile reference frames in their initial positions are plotted in Figure 1, along with the triangle in its initial position. To achieve this, the **quiver3** function is used to plot the unit vectors, while the triangle is represented as a polyline created by summing its vertices.

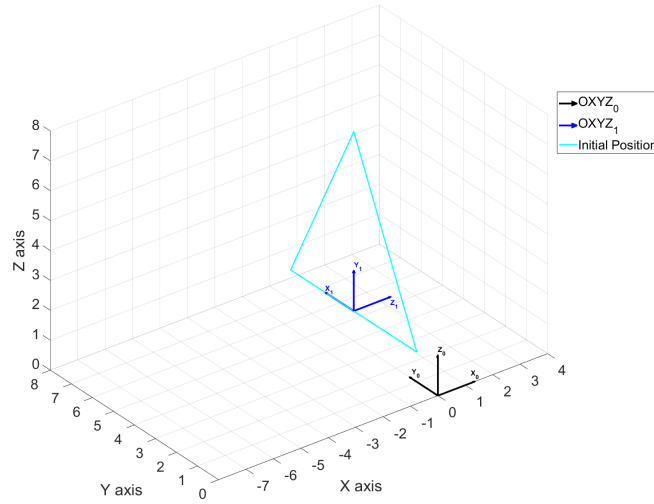


Figure 1: Triangle ABC in its initial position and its related reference system

```

1      % Plot of Unit vectors
2      offset = 1.5;
3
4      % fixed reference frame
5      figure
6
7      fixed_RF_color = 'k';
8      p_i=quiver3(0,0,0,i(1),i(2),i(3),'linewidth',3,Color=fixed_RF_color);hold
9      on % x-axis of reference frame
10     p_j=quiver3(0,0,0,j(1),j(2),j(3),'linewidth',3,Color=fixed_RF_color);hold
11     on % y-axis of reference frame
12     p_k=quiver3(0,0,0,k(1),k(2),k(3),'linewidth',3,Color=fixed_RF_color);hold
13     on % z-axis of reference frame
14
15     text(i(1), i(2), i(3), 'X_0', 'FontSize', 12, 'Color', fixed_RF_color, '
16         FontWeight', 'bold'); % label of x-axis of reference frame
17     text(j(1), j(2), j(3), 'Y_0', 'FontSize', 12, 'Color', fixed_RF_color, '
18         FontWeight', 'bold'); % label of y-axis of reference frame
19     text(k(1), k(2), k(3), 'Z_0', 'FontSize', 12, 'Color', fixed_RF_color, '
20         FontWeight', 'bold'); % label of z-axis of reference frame
21
22     xlabel('X axis')
23     ylabel('Y axis')
24     zlabel('Z axis')
25     axis equal
26     set(gca, 'FontSize', 10)
27
28     % mobile reference frame
29     i10 = [A10(:,1)]*scale;
30     j10 = [A10(:,2)]*scale;
31     k10 = [A10(:,3)]*scale;
32     o10 = [0 4 1];
33
34     mobile_RF_color = 'b';
35     p_i10=quiver3(o10(1),o10(2),o10(3),i10(1),i10(2),i10(3),'linewidth',3,'
36         Color',mobile_RF_color);hold on % x-axis of body frame X1
37     p_j10=quiver3(o10(1),o10(2),o10(3),j10(1),j10(2),j10(3),'linewidth',3,'
38         Color',mobile_RF_color);hold on % y-axis of body frame Y1
39     p_k10=quiver3(o10(1),o10(2),o10(3),k10(1),k10(2),k10(3),'linewidth',3,'
40         Color',mobile_RF_color);hold on % z-axis of body frame Z1
41
42     text(A10o(1,4), A10o(2,4)+offset, A10o(3,4), 'X_1', 'FontSize', 12, 'Color'
43         , mobile_RF_color, 'FontWeight', 'bold'); % label of X1

```

```

34 text(A10o(1,4), A10o(2,4), A10o(3,4)+offset, 'Y_1', 'FontSize', 12, 'Color'
    , mobile_RF_color, 'FontWeight', 'bold'); % label of Y1
35 text(A10o(1,4)+offset, A10o(2,4), A10o(3,4), 'Z_1', 'FontSize', 12, 'Color'
    , mobile_RF_color, 'FontWeight', 'bold'); % label of Z1
36
37 axis equal
38 %% Plot of triangle as a sum of a polyline
39 X = [pA_o(1) pB_o(1) pC_o(1) pA_o(1)];
40 Y = [pA_o(2) pB_o(2) pC_o(2) pA_o(2)];
41 Z = [pA_o(3) pB_o(3) pC_o(3) pA_o(3)];
42
43 pT0 = plot3(X,Y,Z,'color','c',LineWidth=2, MarkerSize=8); hold on
44
45 %% Legend
46 set(gca,'FontSize', 12, 'XLim',[-8 4], 'YLim',[0 8], 'ZLim',[0 8])
47 set(gca,'XTick',(-7:1:4),'YTick',(0:1:8),'ZTick',(0:1:8))
48 set(gcf, 'Color', 'w');
49 lgd=legend([p_i(1) p_i10(1) pT0], 'OXYZ_0','OXYZ_1', 'Initial Position');

```

5 Rotation of the initial triangle of 90° about local axis y_1

To rotate the initial triangle by $\vartheta = 90^\circ$ about the local axis y_1 , the matrix ${}^0\hat{\mathbf{A}}_1$ must be post-multiplied by the rotation matrix $\hat{Rot}(y_1, 90^\circ)$. This procedure yields the new orientation matrix ${}^0\hat{\mathbf{B}}_1$, which describes the orientation and position of the triangle in its final configuration with respect to the fixed reference frame.

To compute the final position vectors of the triangle's vertices in the global system, their initial positions with respect to the mobile reference frame are required. To achieve this, the inverse of the matrix ${}^0\hat{\mathbf{A}}_1$ must be computed and used to pre-multiply the initial vertex position vectors as observed in the global system. Finally, the orientation matrix ${}^0\hat{\mathbf{B}}_1$ can be multiplied by these computed vectors to determine their final positions. Figure 2 illustrates the initial and final positions of the triangle, along with the reference systems.

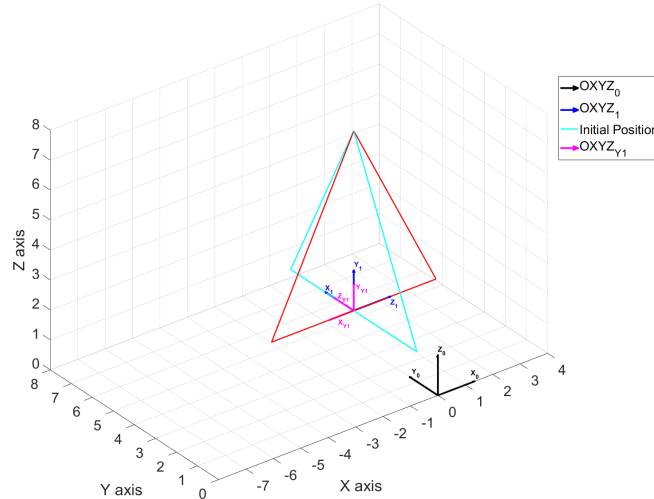


Figure 2: Triangle ABC after a rotation ϑ around its own axis y_1

```

1  %% Rotation of 90 degrees about Y1 axis
2  theta = pi/2;
3  Rot_y1_theta_o = [cos(theta) 0 sin(theta) 0; ...
4                   0 1 0 0; ...
5                   -sin(theta) 0 cos(theta) 0
6                   0 0 0 1];
7

```

```

8      B10o = A10o*Rot_y1_theta_o;      % orientation matrix after the rotation
9
10     %% Triangle after rotation about Y1-axis
11     A10o_1 = inv(A10o);
12     pA1_o = A10o_1*pA_o; % A-vector in initial position in body frame
        system
13     pB1_o = A10o_1*pB_o; % B-vector in initial position in body frame
        system
14     pC1_o = A10o_1*pC_o; % C-vector in initial position in body frame
        system
15
16     pAf_o = B10o*pA1_o; % A-vector in final position in reference frame
        system
17     pBf_o = B10o*pB1_o; % B-vector in final position in reference frame
        system
18     pCf_o = B10o*pC1_o; % C-vector in final position in reference frame
        system
19
20     X_f = [pAf_o(1) pBf_o(1) pCf_o(1) pAf_o(1)];
21     Y_f = [pAf_o(2) pBf_o(2) pCf_o(2) pAf_o(2)];
22     Z_f = [pAf_o(3) pBf_o(3) pCf_o(3) pAf_o(3)];
23
24     % reference system attached to body after rotation about Y1-axis
25     i20 = B10o(1:3,1);
26     j20 = B10o(1:3,2);
27     k20 = B10o(1:3,3);
28     o20 = B10o(1:3,4);
29
30     local_RF_Y1 = 'm';
31     p_i20=quiver3(o20(1),o20(2),o20(3),i20(1),i20(2),i20(3),'linewidth',3,'
        Color',local_RF_Y1);hold on
32     p_j20=quiver3(o20(1),o20(2),o20(3),j20(1),j20(2),j20(3),'linewidth',3,'
        Color',local_RF_Y1);hold on
33     p_k20=quiver3(o20(1),o20(2),o20(3),k20(1),k20(2),k20(3),'linewidth',3,'
        Color',local_RF_Y1);hold on
34
35     text(0-offset, 4, 1, 'X_Y_1', 'FontSize', 12, 'Color', local_RF_Y1, '
        FontWeight', 'bold'); % label of X_Y1
36     text(0, 4, 1+offset, 'Y_Y_1', 'FontSize', 12, 'Color', local_RF_Y1, '
        FontWeight', 'bold'); % label of Y_Y1
37     text(0, 4+offset, 1, 'Z_Y_1', 'FontSize', 12, 'Color', local_RF_Y1, '
        FontWeight', 'bold'); % label of Z_Y1
38
39     pT1 = plot3(X_f,Y_f,Z_f,'r',LineWidth=2, MarkerSize=8);
40
41     %% Legend
42     set(gca,'FontSize', 12, 'XLim',[-8 4], 'YLim',[0 8], 'ZLim',[0 8])
43     set(gca,'XTick',(-7:1:4), 'YTick',(0:1:8), 'ZTick',(0:1:8))
44     set(gcf, 'Color', 'w');
45     lgd=legend([p_i(1) p_i10(1) pT0 p_i20(1) pT1], 'OXYZ_0','OXYZ_1', 'Initial
        Position', ...
46     'XYZ_Y_1','Y_1 rotation');

```

6 Rotation of the initial triangle of -90° about global axis y

In this task, the rotation of the initial triangle by $\alpha = -90^\circ$ about the global axis y is required. Therefore, the matrix ${}^0\hat{\mathbf{A}}_1$, which describes the initial triangle, must be pre-multiplied by the rotation matrix $\hat{Rot}(y, -90^\circ)$. This procedure results in the new orientation matrix ${}^0\hat{\mathbf{C}}_2$, which describes the orientation and position of the triangle in its final configuration with respect to the fixed reference frame.

Figure 3 illustrates the initial and final positions of the triangle, along with the reference systems.

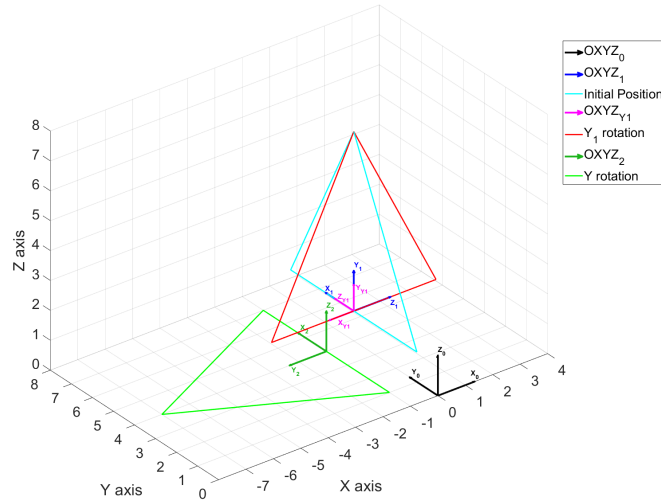


Figure 3: Triangle ABC after a rotation α around global axis y

```

1  %% Rotation of -90 degrees about Y axis from initial position
2  alfa = -pi/2;
3  Rot_y_alfa_o = [cos(alfa) 0 sin(alfa) 0; ...
4                  0 1 0 0; ...
5                  -sin(alfa) 0 cos(alfa) 0; ...
6                  0 0 0 1];
7
8  C20o = Rot_y_alfa_o*A10o; % Pre-multiply Rotation matrix because of
9                             reference frame system
10
11 %% Triangle after rotation about Y-axis
12 pAf2_o = C20o*pA1_o; % A-vector in final position in reference frame
13         system
14 pBf2_o = C20o*pB1_o; % B-vector in final position in reference frame
15         system
16 pCf2_o = C20o*pC1_o; % C-vector in final position in reference frame
17         system
18
19 X_f2 = [pAf2_o(1) pBf2_o(1) pCf2_o(1) pAf2_o(1)];
20 Y_f2 = [pAf2_o(2) pBf2_o(2) pCf2_o(2) pAf2_o(2)];
21 Z_f2 = [pAf2_o(3) pBf2_o(3) pCf2_o(3) pAf2_o(3)];
22
23 pT2 = plot3(X_f2,Y_f2,Z_f2,'g',LineWidth=2, MarkerSize=8);
24
25 % reference system attached to body after rotation about Y1-axis
26 i30 = C20o(1:3,1)*scale;
27 j30 = C20o(1:3,2)*scale;
28 k30 = C20o(1:3,3)*scale;
29 o30 = C20o(1:3,4);
30
31 local_RF_Y = [0.1 0.7 0.1];
32 p_i30=quiver3(o30(1),o30(2),o30(3),i30(1),i30(2),i30(3),'linewidth',3,'
33         Color',local_RF_Y);hold on
34 p_j30=quiver3(o30(1),o30(2),o30(3),j30(1),j30(2),j30(3),'linewidth',3,'
35         Color',local_RF_Y);hold on
36 p_k30=quiver3(o30(1),o30(2),o30(3),k30(1),k30(2),k30(3),'linewidth',3,'
37         Color',local_RF_Y);hold on
38
39 text(-1, 4+offset, 0, 'X_2', 'FontSize', 12, 'Color', local_RF_Y, '
40         FontWeight', 'bold'); % label of X2
41 text(-1-offset, 4, 0, 'Y_2', 'FontSize', 12, 'Color', local_RF_Y, '
42         FontWeight', 'bold'); % label of Y2
43 text(-1, 4, 0+offset, 'Z_2', 'FontSize', 12, 'Color', local_RF_Y, '

```

```

35         FontWeight', 'bold');    % label of Z2
36
37 %% Legend
38 set(gca,'FontSize', 12, 'XLim',[-8 4], 'YLim',[0 8], 'ZLim',[0 8])
39 set(gca,'XTick',(-7:1:4),'YTick',(0:1:8),'ZTick',(0:1:8))
40 set(gcf, 'Color', 'w');
41 lgd=legend([p_i(1) p_i10(1) pT0 p_i20(1) pT1 ...
42 p_i30(1) pT2], 'XYZ_0','XYZ_1', 'Initial Position', ...
43 'XYZ_Y_1','Y_1 rotation','XYZ_2','Y rotation');

```

7 Rotation of the last triangle of 90° about local axis x_2

For the last step, the triangle in its final position needs to be rotated by $\vartheta = 90^\circ$ about the local axis x_2 . This means that the last orientation matrix ${}^0\hat{\mathbf{C}}_2$ must be post-multiplied by the rotation matrix $\hat{Rot}(x_2, 90^\circ)$, and the result will be the new orientation matrix ${}^0\hat{\mathbf{D}}_3$.

Figure 4 shows all the positions of the triangle, along with the global and local reference systems.

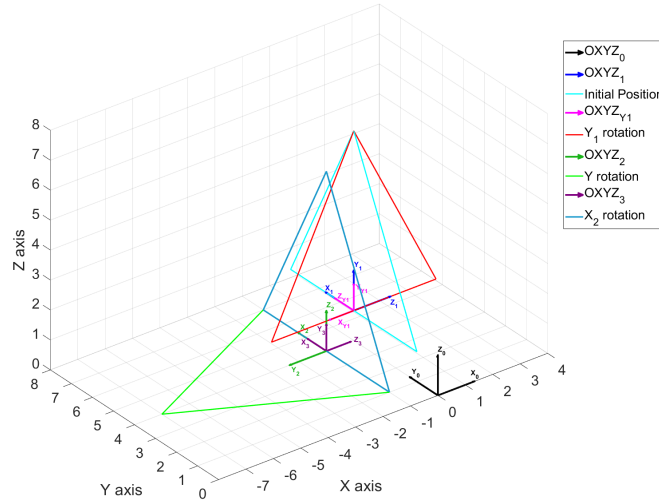


Figure 4: Triangle ABC after a rotation ϑ around local axis x_2

```

1  %% Rotation of 90 degrees about X2 axis from last position
2  Rot_x2_theta_o = [1 0 0 0; ...
3                   0 cos(theta) -sin(theta) 0; ...
4                   0 sin(theta) cos(theta) 0; ...
5                   0 0 0 1];
6
7  D30o = C20o*Rot_x2_theta_o; % Post-multiply Rotation matrix because of
8  % mobile reference system
9
10 %% Triangle after rotation about Y-axis then X2-axis
11 pAf3_o = D30o*pA1_o; % A-vector in final position in reference frame
12 % system
13 pBf3_o = D30o*pB1_o; % A-vector in final position in reference frame
14 % system
15 pCf3_o = D30o*pC1_o; % A-vector in final position in reference frame
16 % system
17
18 X_f3 = [pAf3_o(1) pBf3_o(1) pCf3_o(1) pAf3_o(1)];
19 Y_f3 = [pAf3_o(2) pBf3_o(2) pCf3_o(2) pAf3_o(2)];
20 Z_f3 = [pAf3_o(3) pBf3_o(3) pCf3_o(3) pAf3_o(3)];

```

```

18 pT3 = plot3(X_f3,Y_f3,Z_f3,'color',[0.1 0.6 0.8],LineWidth=2.5,
19         MarkerSize=8);
20
21 % reference system attached to body after rotation about X2-axis
22 i40 = D30o(1:3,1);
23 j40 = D30o(1:3,2);
24 k40 = D30o(1:3,3);
25 o40 = D30o(1:3,4);
26
27 local_RF_X2 = [0.5 0 0.5];
28 p_i40=quiver3(o40(1),o40(2),o40(3),i40(1),i40(2),i40(3),'linewidth',3,'
29         Color',local_RF_X2);hold on
30 p_j40=quiver3(o40(1),o40(2),o40(3),j40(1),j40(2),j40(3),'linewidth',3,'
31         Color',local_RF_X2);hold on
32 p_k40=quiver3(o40(1),o40(2),o40(3),k40(1),k40(2),k40(3),'linewidth',3,'
33         Color',local_RF_X2);hold on
34
35 text(-1, 4+offset, 0, 'X_3', 'FontSize', 12, 'Color', local_RF_X2, '
36         FontWeight', 'bold'); % label of X3
37 text(-1, 4, 0+offset, 'Y_3', 'FontSize', 12, 'Color', local_RF_X2, '
38         FontWeight', 'bold'); % label of Y3
39 text(-1+offset, 4, 0, 'Z_3', 'FontSize', 12, 'Color', local_RF_X2, '
40         FontWeight', 'bold'); % label of Z3
41
42 %% Legend
43 set(gca,'FontSize', 12, 'XLim',[-8 4], 'YLim',[0 8], 'ZLim',[0 8])
44 set(gca,'XTick',(-7:1:4), 'YTick',(0:1:8), 'ZTick',(0:1:8))
45 set(gcf, 'Color', 'w');
46 lgd=legend([p_i(1) p_i10(1) pT0 p_i20(1) pT1 ...
47         p_i30(1) pT2 p_i40(1) pT3], '0XYZ_0','0XYZ_1', 'Initial Position'
48         , ...
49         'XYZ_Y_1','Y_1 rotation','XYZ_2','Y rotation','XYZ_3','X_2 rotation
50         ');

```

8 Conclusions

The exercise successfully demonstrated the implementation of geometric transformations and their visualization in MATLAB, focusing on the rotation of a triangle relative to fixed and mobile reference frames. The main findings are summarized as follows:

- **Positioning Matrix Implementation:** the initial setup of the triangle and its associated reference frames was accurately represented using homogeneous transformation matrices. This ensured that the positions and orientations were properly defined.
- **Visualization of Rotations:** each step of the rotational transformation was plotted, clearly showing the effects of the specified rotations about different axes. This helped in understanding the relationship between the fixed and mobile reference frames.
- **Sequential Rotations:**
 - 90° about the local y_1 -axis
 - -90° about the global y_0 -axis
 - 90° about the local x_2 -axis

The final orientation and position of the triangle aligned with theoretical predictions.

This exercise highlights the importance of homogeneous transformation matrices as a foundational tool for addressing rigid body transformation problems.



**Politecnico
di Torino**

Exercise 3: SCARA robot

Master's Degree in Automotive Engineering

Maurizio Pio Vergara: `s346643@studenti.polito.it`

Loris Fonseca: `s342696@studenti.polito.it`

Renis Shtylla: `s344685@studenti.polito.it`

a.a 2024/2025

Contents

1	Objective	3
2	Methodology	3
3	Initial and final positions plot	3
4	End effector O_3 trajectory	8
5	Conclusions	9

1 Objective

The aim of this exercise is to study the behavior of the SCARA robot based on the given joint trajectories. This robot has two revolute joints and one prismatic joint. The reference frame for each joint is defined in the exercise's plot, following the Denavit-Hartenberg convention, as show in Figure 1.

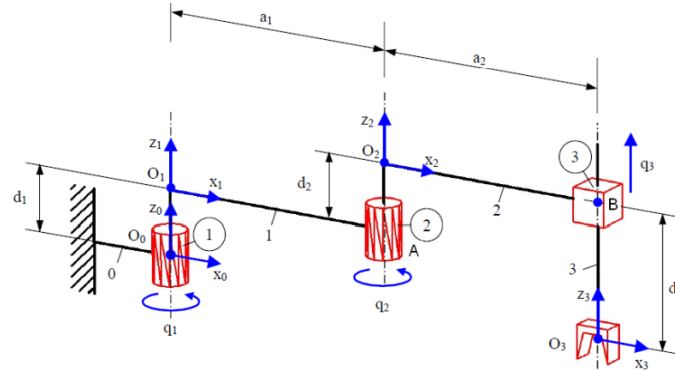


Figure 1: SCARA robot sketch

2 Methodology

Firstly, the joints trajectories need to be imported into MATLAB project using the "LOAD" function. Secondly, joints parameters can be specified by creating column vectors with values extracted from the 'joints.mat' matrix file. Finally, the Denavit-Hartenberg parameters, reported in Table 1, can be defined. A vectorial form has been chosen to simplify the MATLAB code.

```

1  clc
2  clear
3  close all
4  %% Load data
5  load ('joints.mat')
6  q1 = M(:,1);
7  q2 = M(:,2);
8  q3 = M(:,3);
9
10                                     %% Initial Position of the SCARA robot
11
12  % D-H parameters
13  alfa = [0 0 0];
14  a = [0 50 10];
15  d = [25 15 q3(1)-10];
16  teta = [q1(1) q2(1) 0];

```

i	α [rad]	a [mm]	d [mm]	θ [rad]
1	0	0	25	q_1
2	0	50	15	q_2
3	0	10	$q_3 - 10$	0

Table 1: Denavit-Hartenberg parameters

3 Initial and final positions plot

To obtain the orientation matrices, the function *denhar_en* is used. This function takes the four D-H parameters as input and outputs a matrix. Having the orientation matrix makes it possible to determine the origin of each reference frame relative to the fixed frame. The positions of points A and B are provided in the exercise's plot and can be expressed relative to their body

frames. These positions can then be expressed in fixed frame by **pre-multiplying** them with the orientation matrices ${}^0\hat{\mathbf{A}}_2$ or ${}^0\hat{\mathbf{A}}_3$.

The functions *quiver3* and *plot3* are used to visualize the reference systems axes and the wire frame of the SCARA robot. The function *joint_rev_01*, provided in the exercise, is used to plot revolute and prismatic joints. In Figure 2 the initial position of the robot is shown, while in Figure 3 both initial and final positions are shown.

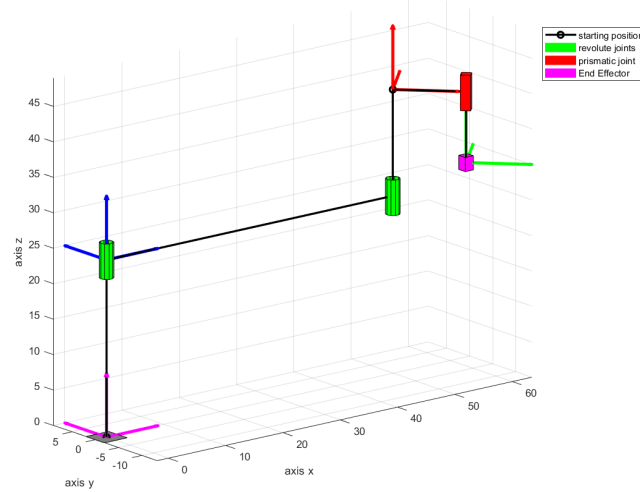


Figure 2: SCARA robot initial position

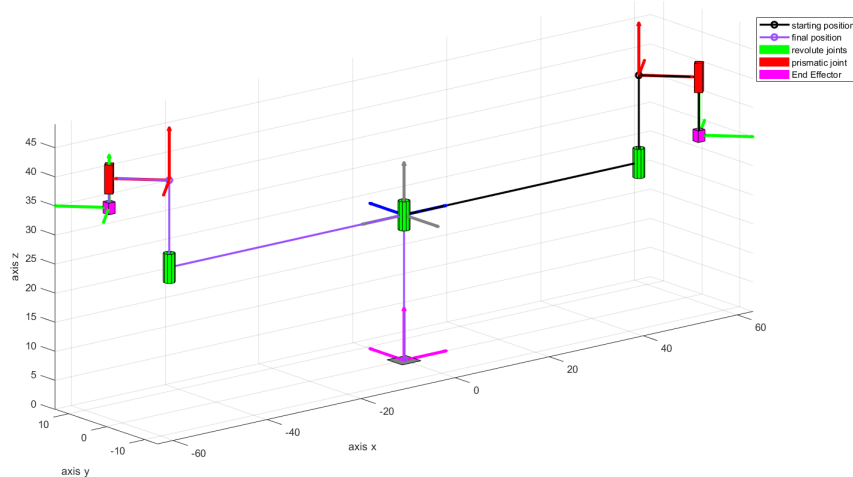


Figure 3: Initial position and final positions

```

1  %% Orientation Matrices
2  A10 = denhar_en(alfa(1),a(1),d(1),teta(1));
3  A21 = denhar_en(alfa(2),a(2),d(2),teta(2));
4  A32 = denhar_en(alfa(3),a(3),d(3),teta(3));
5  A20 = A10*A21;
6  A30 = A20*A32;
7
8  %% Origins of the systems
9  p0 = [0 0 0 1];
10 p01 = A10(:,4);
11 p02 = A20(:,4);
12 p03 = A30(:,4);
13
14 pA1initial = [a(2),0,0,1]';
15 pB2initial = [a(3),0,0,1]';
16
17 pA0 = A10*pA1initial;

```

```

18 pB0 = A20*pB2initial;
19
20 %% reference frame zero
21 S = 10;
22 i0 = [1 0 0]*S;
23 j0 = [0 1 0]*S;
24 k0 = [0 0 1]*S;
25
26 x0 = quiver3(0,0,0,i0(1),i0(2),i0(3),'m','linewidth',3);hold on
27 y0 = quiver3(0,0,0,j0(1),j0(2),j0(3),'m','linewidth',3);hold on
28 z0 = quiver3(0,0,0,k0(1),k0(2),k0(3),'m','linewidth',3);hold on
29
30 %% reference frame one
31 x1 = quiver3(p01(1),p01(2),p01(3),A10(1,1)*S,A10(2,1)*S,A10(3,1)*S,'b',
32 ...
33 'linewidth',3);hold on
34 y1 = quiver3(p01(1),p01(2),p01(3),A10(1,2)*S,A10(2,2)*S,A10(3,2)*S,'b',
35 ...
36 'linewidth',3);hold on
37 z1 = quiver3(p01(1),p01(2),p01(3),A10(1,3)*S,A10(2,3)*S,A10(3,3)*S,'b',
38 ...
39 'linewidth',3);hold on
40
41 %% reference frame two
42 x2 = quiver3(p02(1),p02(2),p02(3),A20(1,1)*S,A20(2,1)*S,A20(3,1)*S,'r',
43 ...
44 'linewidth',3);hold on
45 y2 = quiver3(p02(1),p02(2),p02(3),A20(1,2)*S,A20(2,2)*S,A20(3,2)*S,'r',
46 ...
47 'linewidth',3);hold on
48 z2 = quiver3(p02(1),p02(2),p02(3),A20(1,3)*S,A20(2,3)*S,A20(3,3)*S,'r',
49 ...
50 'linewidth',3);hold on
51
52 %% reference frame three
53 x3 = quiver3(p03(1),p03(2),p03(3),A30(1,1)*S,A30(2,1)*S,A30(3,1)*S,'g',
54 ...
55 'linewidth',3);hold on
56 y3 = quiver3(p03(1),p03(2),p03(3),A30(1,2)*S,A30(2,2)*S,A30(3,2)*S,'g',
57 ...
58 'linewidth',3);hold on
59 z3 = quiver3(p03(1),p03(2),p03(3),A30(1,3)*S,A30(2,3)*S,A30(3,3)*S,'g',
60 ...
61 'linewidth',3);hold on
62
63 %% plot frame in initial position
64 start = plot3([p0(1),p01(1),pA0(1),p02(1),pB0(1),p03(1)], ...
65 [p0(2),p01(2),pA0(2),p02(2),pB0(2),p03(2)], ...
66 [p0(3),p01(3),pA0(3),p02(3),pB0(3),p03(3)], '-ok','linewidth',2);
67 hold on
68
69 %% plot joints
70 % Revolute Joint matrix is a homogeneous 4x4 with a 3x3 unity matrix
71 indicating the
72 % directions of the axes, and 0(i) as its origin.
73
74 Ajoint1 = eye(4);
75 Ajoint1(:,4) = p01; % homogeneous unity matrix with origin in p01
76 Ajoint2 = eye(4);
77 Ajoint2(:,4) = pA0; % homogeneous unity matrix with origin in pA0
78 Ajoint3 = eye(4);
79 Ajoint3(:,4) = pB0; % homogeneous unity matrix with origin in pB0
80
81 R=1; H=5; N=10;
82
83 rev1 = joint_rev_01(R,H,N,Ajoint1,'g'); % revolute joint in p01

```



```

73 rev2 = joint_rev_01(R,H,N,Ajoint2,'g');      % revolute joint in pA0
74 N = 4;
75 prs3 = joint_rev_01(R,H,N,Ajoint3,'red');    % prismatic joint in pB0
76 H = 2;
77 rbt4 = joint_rev_01(R,H,N,A30,'m');          % End Effector
78
79 %% Plot base
80 fill3([-2 2 2 -2],[-2 -2 2 2],[0 0 0 0],[.5 .5 .5])
81
82 daspect([1 1 1]) % fix the ratio between axis dimensions
83 view([-39 16])   % specify camera position as [azimuth, elevation]
84
85 %% Final Position of the SCARA robot
86 % D-H parameters
87 alfa_f = [0 0 0];
88 a_f = [0 50 10];
89 d_f = [25 15 q3(end)-10];
90 teta_f = [q1(end) q2(end) 0];
91
92 %% Orientation Matrices
93 A10_f = denhar_en(alfa_f(1),a_f(1),d_f(1),teta_f(1));
94 A21_f = denhar_en(alfa_f(2),a_f(2),d_f(2),teta_f(2));
95 A32_f = denhar_en(alfa_f(3),a_f(3),d_f(3),teta_f(3));
96 A20_f = A10_f*A21_f;
97 A30_f = A20_f*A32_f;
98
99 %% Origins of the systems
100 p01_f = A10_f(:,4);
101 p02_f = A20_f(:,4);
102 p03_f = A30_f(:,4);
103
104 pA0_f = A10_f*pA1initial;
105 pB0_f = A20_f*pB2initial;
106
107 %% reference frame one
108 x1_f = quiver3(p01_f(1),p01_f(2),p01_f(3),A10_f(1,1)*S,A10_f(2,1)*S,
109               A10_f(3,1)*S, ...
110               'Color',[0.5 0.5 0.5],'linewidth',3);hold on
111 y1_f = quiver3(p01_f(1),p01_f(2),p01_f(3),A10_f(1,2)*S,A10_f(2,2)*S,
112               A10_f(3,2)*S, ...
113               'Color',[0.5 0.5 0.5],'linewidth',3);hold on
114 z1_f = quiver3(p01_f(1),p01_f(2),p01_f(3),A10_f(1,3)*S,A10_f(2,3)*S,
115               A10_f(3,3)*S, ...
116               'Color',[0.5 0.5 0.5],'linewidth',3);hold on
117
118 %% reference frame two
119 x2_f = quiver3(p02_f(1),p02_f(2),p02_f(3),A20_f(1,1)*S,A20_f(2,1)*S,
120               A20_f(3,1)*S,'r', ...
121               'linewidth',3);hold on
122 y2_f = quiver3(p02_f(1),p02_f(2),p02_f(3),A20_f(1,2)*S,A20_f(2,2)*S,
123               A20_f(3,2)*S,'r', ...
124               'linewidth',3);hold on
125 z2_f = quiver3(p02_f(1),p02_f(2),p02_f(3),A20_f(1,3)*S,A20_f(2,3)*S,
126               A20_f(3,3)*S,'r', ...
127               'linewidth',3);hold on
128
129 %% reference frame three
130 x3_f = quiver3(p03_f(1),p03_f(2),p03_f(3),A30_f(1,1)*S,A30_f(2,1)*S,
131               A30_f(3,1)*S,'g', ...
132               'linewidth',3);hold on
133 y3_f = quiver3(p03_f(1),p03_f(2),p03_f(3),A30_f(1,2)*S,A30_f(2,2)*S,
134               A30_f(3,2)*S,'g', ...
135               'linewidth',3);hold on
136 z3_f = quiver3(p03_f(1),p03_f(2),p03_f(3),A30_f(1,3)*S,A30_f(2,3)*S,
137               A30_f(3,3)*S,'g', ...
138               'linewidth',3);hold on

```

```

130
131     %% plot frame in final position
132     final = plot3([p0(1),p01_f(1),pA0_f(1),p02_f(1),pB0_f(1),p03_f(1)], ...
133     [p0(2),p01_f(2),pA0_f(2),p02_f(2),pB0_f(2),p03_f(2)], ...
134     [p0(3),p01_f(3),pA0_f(3),p02_f(3),pB0_f(3),p03_f(3)], ...
135     '-ok','linewidth',2,'Color',[0.6 0.3 1]);hold on
136
137     %% plot joints
138     Ajoint2_f = eye(4);
139     Ajoint2_f(:,4) = pA0_f;
140     Ajoint3_f = eye(4);
141     Ajoint3_f(:,4) = pB0_f;
142
143     R=1; H=5; N=10;
144
145     rev2_f = joint_rev_01(R,H,N,Ajoint2_f,'g');      % revolute joint in
146     pA0_f
147     N = 4;
148     prs3_f = joint_rev_01(R,H,N,Ajoint3_f,'red');    % prismatic joint in
149     pB0_f
150     H = 2;
151     rbt4_f = joint_rev_01(R,H,N,A30_f,'m');          % End Effector in final
152     position
153
154     %% Legend
155     % "legend" does not allow DOUBLE elements.
156     % "patch" creates a block.
157     % patch block is invisible in the final plot and it is used to
158     show
159     % joint's color with its name in the legend,
160     rev = patch('XData', [0 1 1 0], 'YData', [0 0 1 1], 'FaceColor', 'g',
161     ...
162     'EdgeColor', 'none', 'DisplayName','revolute joints');
163     prism = patch('XData', [0 1 1 0], 'YData', [0 0 1 1], 'FaceColor', 'r',
164     ...
165     'EdgeColor', 'none', 'DisplayName','prismatic joint');
166     EndE = patch('XData', [0 1 1 0], 'YData', [0 0 1 1], 'FaceColor', 'm',
167     ...
168     'EdgeColor', 'none', 'DisplayName','End Effector');
169
170     xlabel('axis x')
171     ylabel('axis y')
172     zlabel('axis z')
173     axis equal
174     grid on
175     hold on
176     lgd = legend([start, final, rev, prism, EndE], 'starting position', ...
177     'final position', 'revolute joints', 'prismatic joint', 'End Effector'
178     );
179
180     function A=denhar_en(alfa,a,d,teta)
181
182     A=[cos(teta)          -sin(teta)          0          a
183     sin(teta)*cos(alfa)   cos(teta)*cos(alfa)  -sin(alfa)  -d*sin(
184     alfa)
185     sin(teta)*sin(alfa)   cos(teta)*sin(alfa)   cos(alfa)   d*cos(alfa
186     )
187     0                      0                      0          1];
188
189     end
190     function [h]=joint_rev_01(R,H,N,A,col)
191
192     [xc,yc,zc1] = cylinder(R, N);
193     [h] = [xc,yc,zc1];
194     zc=zc1*H-H/2;

```

```

186 base1=A*[xc(1,:); yc(1,:);zc(1,:);ones(1,N+1)];
187 base2=A*[xc(2,:); yc(2,:);zc(2,:);ones(1,N+1)];
188 xc0=[base1(1,:);base2(1,:)];
189 yc0=[base1(2,:);base2(2,:)];
190 zc0=[base1(3,:);base2(3,:)];
191 surface(xc0,yc0,zc0,'FaceColor', col)
192
193 end

```

4 End effector O_3 trajectory

A "for" cycle is used to do a reiterative procedure, which can compute the position of the SCARA robot per each joint component defined in the matrix M '**joints.mat**', in order to plot the trajectory of the end effector O_3 . Each position of the origin O_3 can be acquired and saved in a 4×1000 vector, which will be used to visualize the trajectory. In Figure 4 the end effector trajectory is shown.

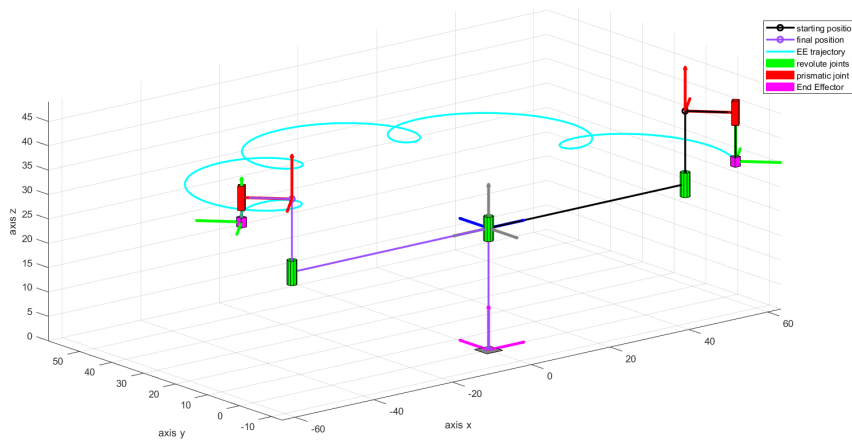


Figure 4: End effector O_3 trajectory

```

1  %% Plot of the end effector p03 trajectory
2  O3traj = ones(4,1000);
3  for i=1:size(M)
4
5      A10inst = denhar_en(alfa(1),a(1),d(1),q1(i));
6      A21inst = denhar_en(alfa(2),a(2),d(2),q2(i));
7      A32inst = denhar_en(alfa(3),a(3),q3(i)-10,teta(3));
8      A20inst = A10inst*A21inst;
9      A30inst = A10inst*A21inst*A32inst;
10     O3traj(:,i)=A30inst(:,4);
11
12 end
13 trajectory = plot3(O3traj(1,:),O3traj(2,:),O3traj(3:,:), 'c', 'linewidth'
14                 ,2);
15
16 %% Legend
17 % "legend" does not allow DOUBLE elements.
18 % "patch" creates a block.
19 % patch block is invisible in the final plot and it is used to show
20 % a the joint color with its name in the legend,
21 rev = patch('XData', [0 1 1 0], 'YData', [0 0 1 1], 'FaceColor', 'g', ...
22             'EdgeColor', 'none', 'DisplayName','revolute joints');
23 prism = patch('XData', [0 1 1 0], 'YData', [0 0 1 1], 'FaceColor', 'r', ...
24              'EdgeColor', 'none', 'DisplayName','prismatic joint');
25 EndE = patch('XData', [0 1 1 0], 'YData', [0 0 1 1], 'FaceColor', 'm', ...
26             'EdgeColor', 'none', 'DisplayName','End Effector');
27 set(gcf, 'Color', 'white');

```

```

28 xlabel('axis x')
29 ylabel('axis y')
30 zlabel('axis z')
31 axis equal
32 grid on
33 hold on
34 lgd = legend([start, final, trajectory, rev, prism, EndE], 'starting
    position', ...
35    'final position', 'EE trajectory', 'revolute joints', 'prismatic joint', '
    End Effector');

```

5 Conclusions

This report effectively analyzed the behavior of a SCARA robot using the provided joint trajectories. By leveraging MATLAB for simulation, the robot's motion was accurately modeled, visualized, and evaluated. The following conclusions can be drawn from the study:

- **Initial and Final Configurations:** the SCARA robot's initial and final configurations were successfully plotted using the Denavit-Hartenberg convention. This allowed for precise orientation and position determination of all joints and the end effector relative to the fixed frame.
- **Trajectory Analysis:** the iterative computation and plotting of the end effector's trajectory showcased a smooth and consistent motion path. This demonstrates the accuracy of the defined kinematic parameters and the reliability of the MATLAB implementation.
- **Performance of Joint Types:** the revolute and prismatic joints were well-illustrated, with the end effector achieving the desired position at the end of the motion sequence.



**Politecnico
di Torino**

Exercise 4: 5 D.O.F. robot on MATLAB

Master's Degree in Automotive Engineering

Maurizio Pio Vergara: `s346643@studenti.polito.it`

Loris Fonseca: `s342696@studenti.polito.it`

Renis Shtylla: `s344685@studenti.polito.it`

a.a 2024/2025

Contents

1	Objective	3
2	Methodology	3
3	Reference frames and joint parameters	3
4	Wire frame and trajectories plot	5
5	Kinematic evaluation	5
6	Dynamic evaluation	8
7	Comparison of the results with <i>Adams</i>	9
8	Conclusions	10
9	Appendix	11

1 Objective

In this exercise, a 5-axis robot with an articulated arm (3 degrees of freedom), a wrist (2 degrees of freedom), and attached payload is considered. The goal is to plot the wire frame model of the robot, with reference frames defined according to the Denavit-Hartenberg convention. Moreover, given the joint parameters and the initial and final positions of the robot, the trajectories of the wrist center point and the end effector are plotted. Additionally, kinematic and inverse dynamic procedures are applied to plot the speeds and accelerations of frame 5, as well as the forces and torques exerted by base 0 on the floor during the movement.

2 Methodology

The exercise is solved by using the given figure of the robot with its dimensions and masses, the file **'trajectory.mat'** which contains the joint parameters, and the given MATLAB functions **'denhar_en01.mat'**, **'kinem_en02.mat'**, **'dynam_en02.mat'** and **'joint_rev_01.mat'**. MATLAB code is fully reported in the Appendix at the end of the document.

3 Reference frames and joint parameters

The given figure of the robot depicts the links and joint positions, representing a configuration with null joint degrees of freedom. The reference frames of the links, defined according to Denavit-Hartenberg convention, are shown in Figure 1. The parameters related to Denavit-Hartenberg convention are reported in Table 1.

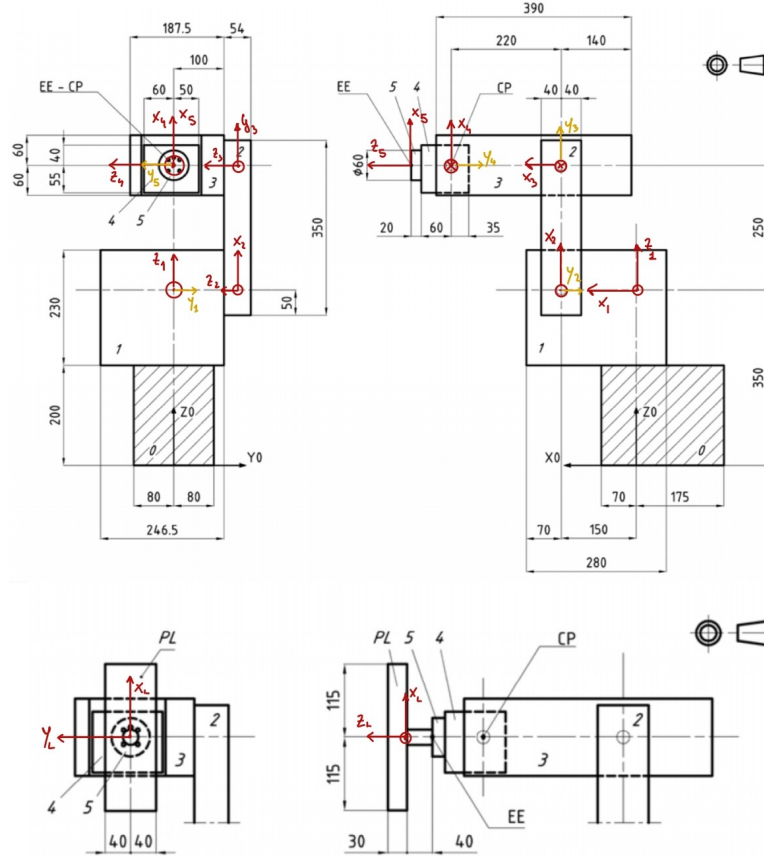


Figure 1: 5-axes robot reference frames

i	α [rad]	a [m]	d [m]	θ [rad]
1	0	0	0.350	q_1
2	$\pi/2$	0.150	-0.100	$q_2 + \pi/2$
3	0	0.250	0	$q_3 - \pi/2$
4	0	0.220	0.127	$q_4 + \pi/2$
5	$\pi/2$	0	0.080	q_5

Table 1: Denavit-Hartenberg parameters

By loading the file **'trajectory.mat'** it is possible to plot angles, speeds and accelerations in joints in time domain. The results are show in Figure 2, Figure 3 and Figure 4.

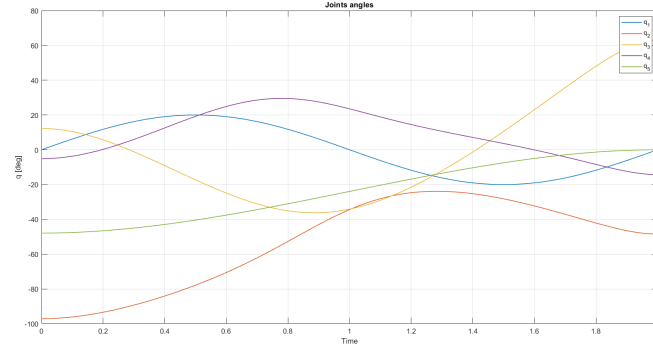


Figure 2: Angles in joints

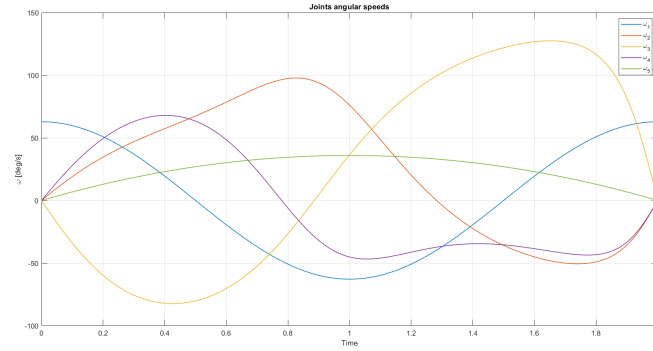


Figure 3: Speeds in joints

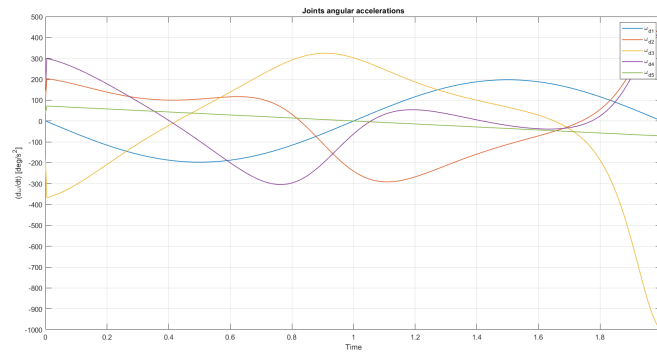


Figure 4: Accelerations in joints

4 Wire frame and trajectories plot

The given joint parameters are expressed in *degrees*, so the conversion in *radians* is needed for the calculus. Once defined Denavit-Hartenberg matrix in MATLAB, the function '**denhar_en01**' can be used to compute the orientation matrices. The origins of the reference frames can be extracted by these matrices, as they correspond to the last column of each matrix. Finally, the functions '**plot3**' and '**joint_rev_01**' are used to plot the wire frame model of the robot in its initial and final positions, as shown in Figure 5.

To obtain and plot the trajectories of the wrist center point (located in O_4) and the end effector (located in O_5) a **for** loop is employed. The coordinates of these points over time are saved in matrices with 4 rows and as many columns as the length of the given time vector.

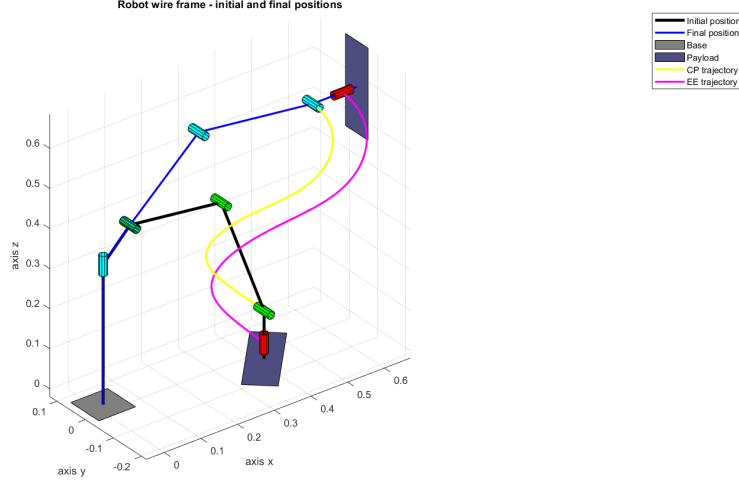


Figure 5: Robot wire frame and trajectories

5 Kinematic evaluation

The given kinematic function is imported into MATLAB to calculate the speeds and accelerations of the entire system. The kinematic evaluation follows a direct procedure, starting with base 0. It is essential to determine the speeds and accelerations of the base, which, according to the exercise, is fixed and has null speeds and accelerations. These parameters serve as inputs, along with the orientation matrix 1A_0 , the position vector of origin 1 relative to frame 0 (expressed in frame 0), the degrees of freedom for speed and acceleration of joint 1, the z-axis of link 1 (expressed in its reference frame), and the joint parameter δ (equal to 0 for revolute joints and 1 for prismatic joints). The outputs of the function are the angular velocity and acceleration, as well as the linear velocities and accelerations of the origin and the center of mass, expressed in local frame 1.

This is an iterative procedure, where the outputs from one step serve as inputs for the next joint's calculations. The final equation in the loop corresponds to the payload, as it represents the last frame of the robot. The **for** loop ends after 1001 times, according to the length of the time vector *timevec*, which represents the time coordinate. As requested, the primary focus is on frame 5, with the results shown in Figure 6 and Figure 7. To facilitate comparison, the speeds and accelerations of the preceding frames are displayed in Figure 8, Figure 9, Figure 10, and Figure 11.

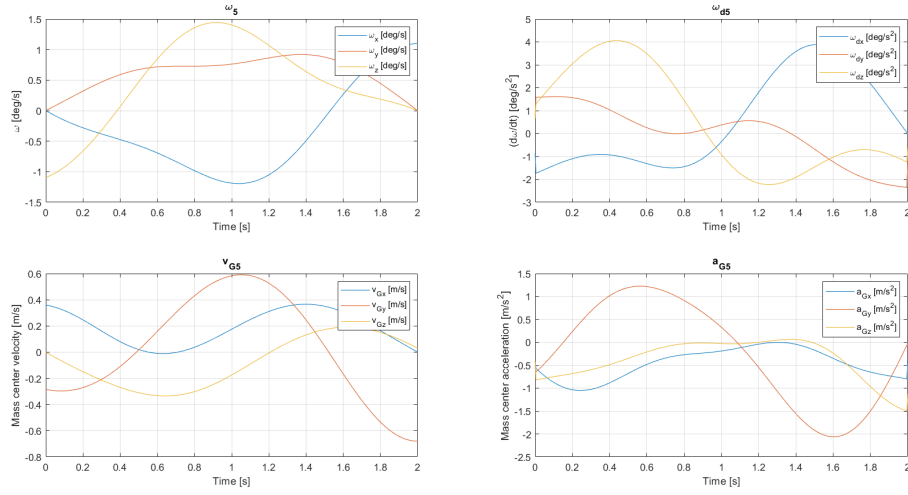


Figure 6: Speeds and accelerations of the mass centre of Frame 5

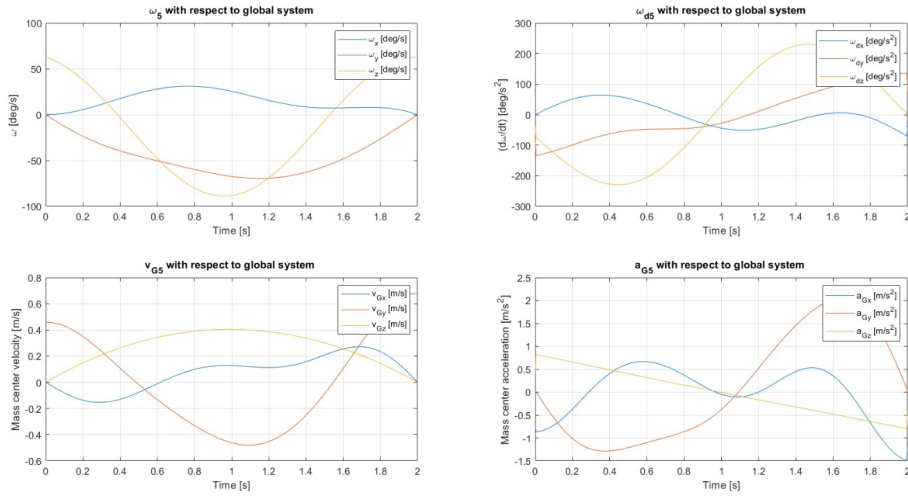


Figure 7: Angular speed and acceleration of Frame 5 with respect to the global system

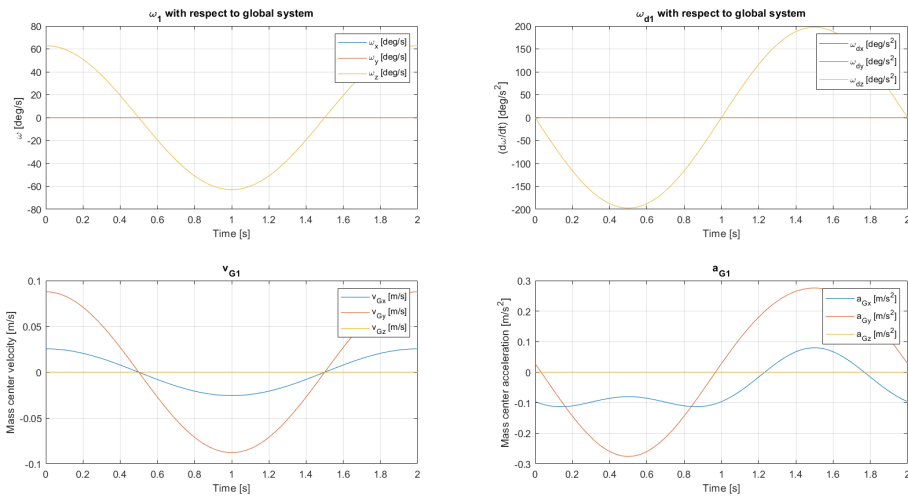


Figure 8: Speeds and accelerations of the mass centre of Frame 1 with respect to the global system

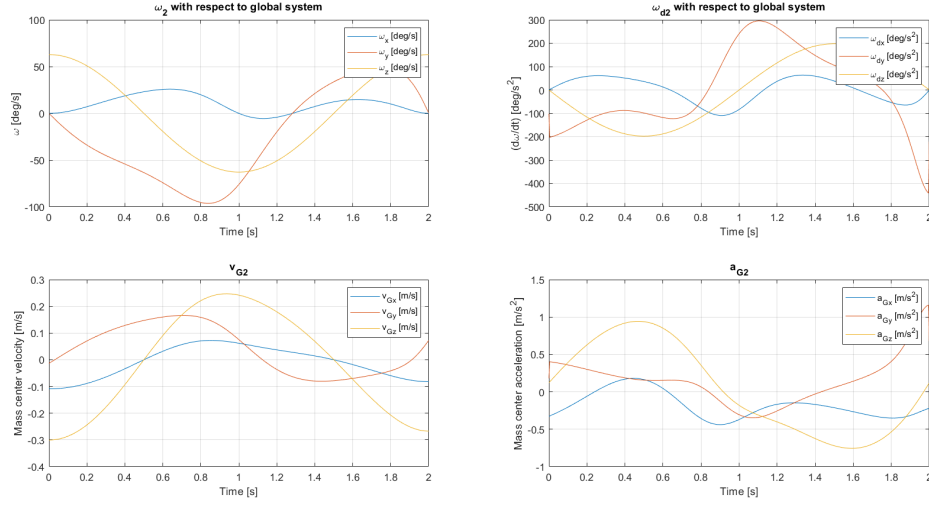


Figure 9: Speeds and accelerations of the mass centre of Frame 2 with respect to the global system

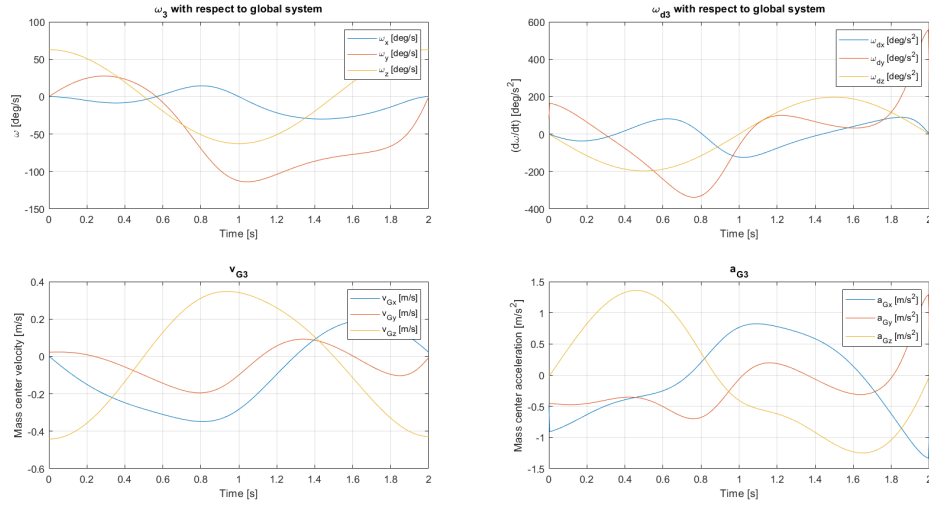


Figure 10: Speeds and accelerations of the mass centre of Frame 3 with respect to the global system

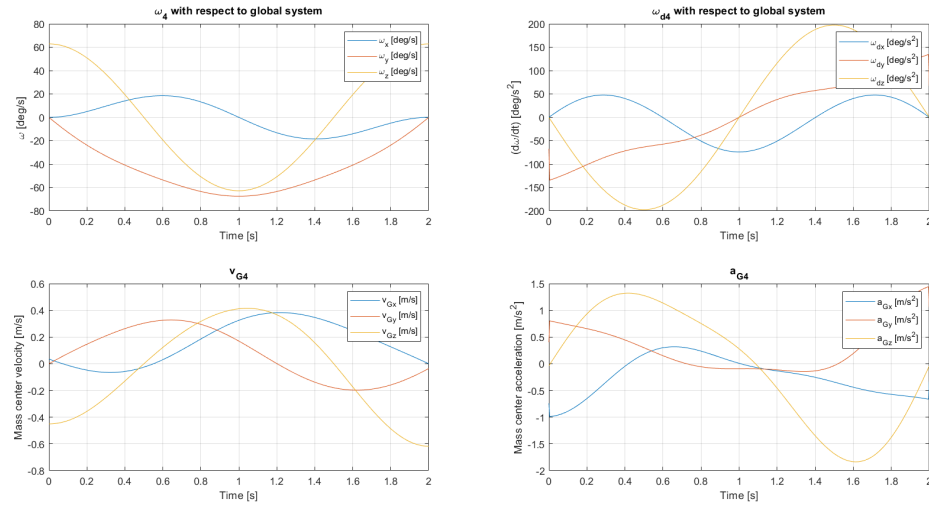


Figure 11: Speeds and accelerations of the mass centre of Frame 4 with respect to the global system

6 Dynamic evaluation

For the dynamic evaluation, dimensions and masses of the robot and of the payload are given. It is essential to compute the inertial matrix of each body, according to its parameters and shape. Parallelepiped shape is considered for each body except frame 5, which is a cylinder. Homogeneous distributed masses are assumed for each body and payload. The dynamic problem follow an indirect procedure, so instead of starting from the base 0, the payload parameters are the first inputs of the first equation inside the **for** loop. The outputs of the function are the force and moment exerted on the considered joint by the previous link, expressed in the joint's local frame. The force and the moment exerted by base 0 to the floor are equal to the opposite of the ones computed for the joint 0. The actuator torques, as well as forces and torques trasmitted to the floor by base 0, are displayed in Figure 12 and Figure 13.

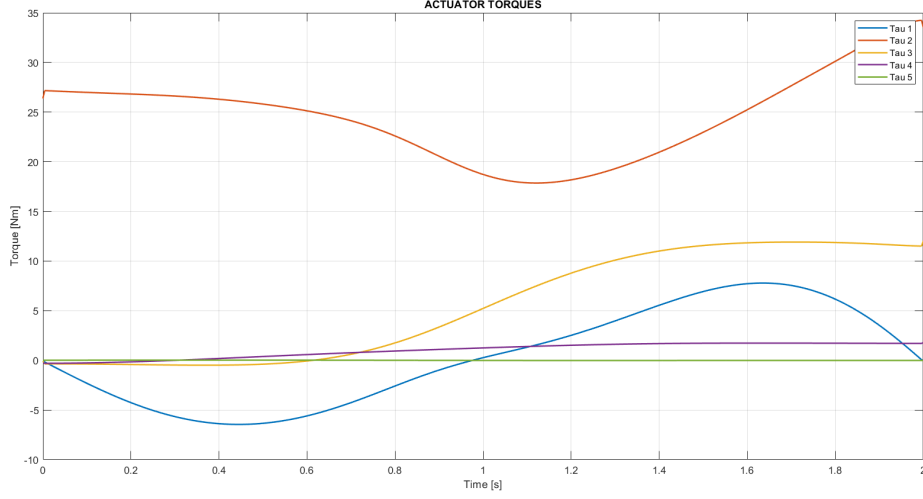


Figure 12: Actuator torques in local reference system

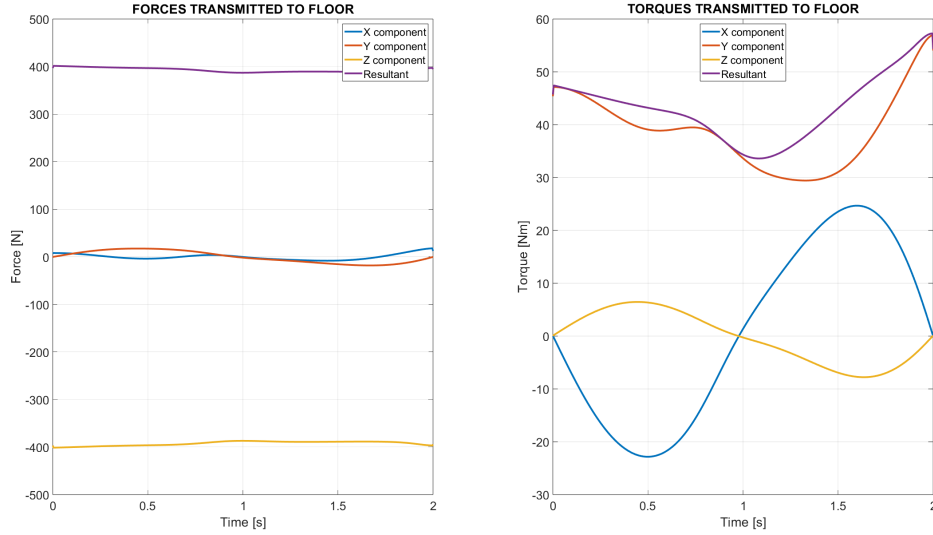
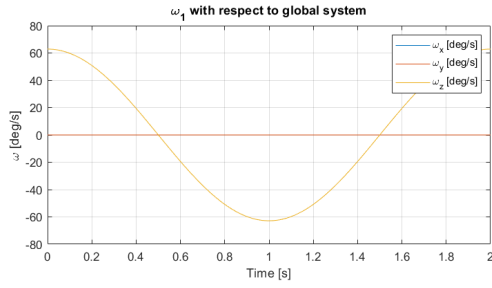


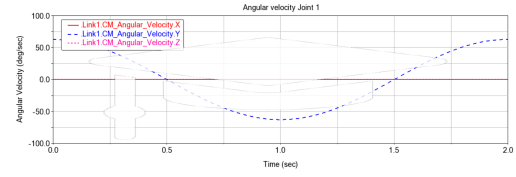
Figure 13: Forces and torques transmitted to the floor by base 0

7 Comparison of the results with *Adams*

In this section the kinematic results computed on MATLAB are compared with the ones obtained using **Adams**. In the following figures, the angular velocity of each joint about its center of mass with respect to the global frame is displayed in both MATLAB and Adams plots.

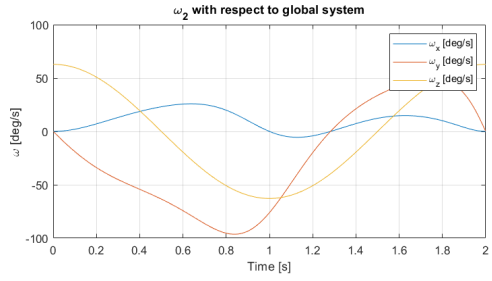


(a) MATLAB

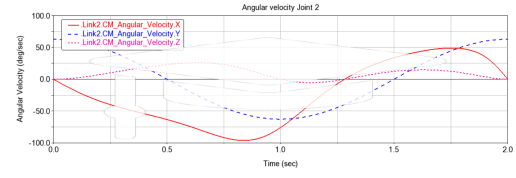


(b) Adams

Figure 14: Angular velocity joint 1

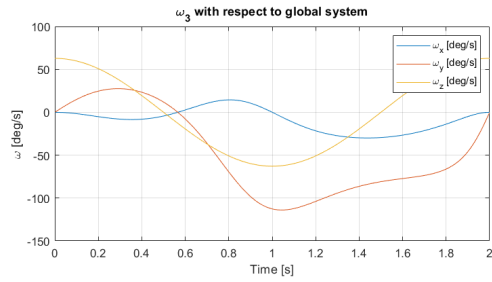


(a) MATLAB

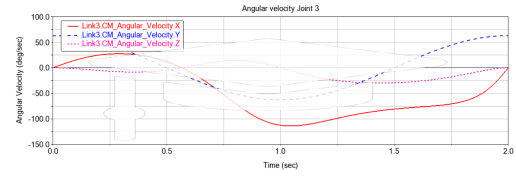


(b) Adams

Figure 15: Angular velocity joint 2

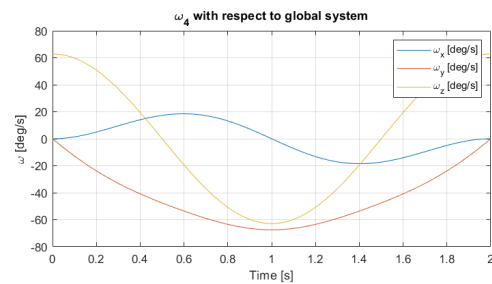


(a) MATLAB

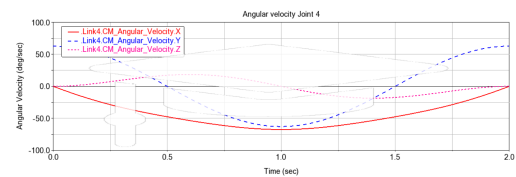


(b) Adams

Figure 16: Angular velocity joint 3

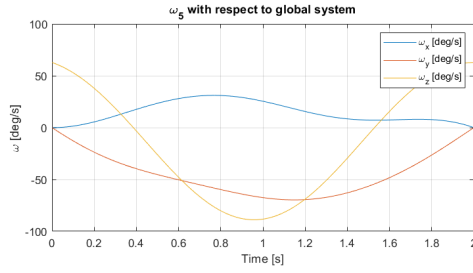


(a) MATLAB

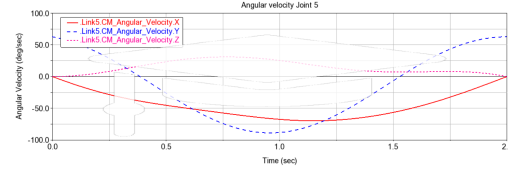


(b) Adams

Figure 17: Angular velocity joint 4



(a) MATLAB



(b) Adams

Figure 18: Angular velocity joint 5

It is noticeable that the angular velocity results were consistent between the two softwares. The only difference lies in the reference axes, as, for instance, the y-axis in Adams corresponds to the z-axis in MATLAB.

8 Conclusions

Below are the key conclusions drawn from the analysis and evaluation of the 5-DOF robotic system.

- **Objective Accomplishment:** the report successfully demonstrates the analysis of a 5-DOF robotic system, covering wireframe plotting, trajectory visualization, kinematic analysis, and dynamic evaluation. The methodologies applied achieved the intended goals outlined in the objective.
- **Kinematic Analysis:** the kinematic evaluation shows the speeds and accelerations of the robot's components, particularly the frame 5 end effector, confirming the accuracy of motion calculations. The iterative approach validates the effectiveness of the applied MATLAB functions and procedures.
- **Dynamic Analysis:** the dynamic evaluation computed the forces and moments transmitted across the robotic structure, including those at the base. The results align with the expected values, providing insight into the actuator torques and ground interactions.
- **Comparison with Adams:** results derived from MATLAB were cross-validated with the Adams simulation tool. Both methods yielded consistent angular velocity profiles for all joints, with minor differences due to reference axis conventions.

9 Appendix

```
1 clc
2 clear
3 close all
4
5     %% Loading joints parameters
6 load('trajectory.mat')
7
8 %% Plot angles in joints systems
9 figure()
10
11 plot(timevec,q1vec,timevec,q2vec,timevec,q3vec,timevec,q4vec,timevec,q5vec,
12     'linewidth',1)
13 grid on
14 xlabel('Time')
15 ylabel('q [deg]')
16 title('Joints angles')
17 set(gcf, 'Color', 'w');
18 legend('q_1','q_2','q_3','q_4','q_5')
19
20 %% Plot angular velocities in joints systems
21 figure()
22 plot(timevec,q1dvec,timevec,q2dvec,timevec,q3dvec,timevec,q4dvec,timevec,
23     q5dvec,'linewidth',1)
24 grid on
25 xlabel('Time')
26 ylabel('\omega [deg/s]')
27 title('Joints angular speeds')
28 set(gcf, 'Color', 'w');
29 legend('\omega_1','\omega_2','\omega_3','\omega_4','\omega_5')
30
31 %% Plot angular accelerations in joints systems
32 figure()
33 plot(timevec,q1ddvec,timevec,q2ddvec,timevec,q3ddvec,timevec,q4ddvec,
34     timevec,q5ddvec,'linewidth',1)
35 grid on
36 xlabel('Time')
37 ylabel('(d\omega/dt) [deg/s^2]')
38 title('Joints angular accelerations')
39 set(gcf, 'Color', 'w');
40 set(gca, 'YTick', (-1000:100:500))
41 legend('\omega_d_1','\omega_d_2','\omega_d_3','\omega_d_4','\omega_d_5')
42
43 %% Degree to radians conversion
44     % useful for further calculations
45
46 q1 = (pi/180)*q1vec;
47 q2 = (pi/180)*q2vec;
48 q3 = (pi/180)*q3vec;
49 q4 = (pi/180)*q4vec;
50 q5 = (pi/180)*q5vec;
51
52 q1d = (pi/180)*q1dvec;
53 q2d = (pi/180)*q2dvec;
54 q3d = (pi/180)*q3dvec;
55 q4d = (pi/180)*q4dvec;
56 q5d = (pi/180)*q5dvec;
57
58 qLd = q5d;
59
60 q1dd = (pi/180)*q1ddvec;
61 q2dd = (pi/180)*q2ddvec;
62 q3dd = (pi/180)*q3ddvec;
63 q4dd = (pi/180)*q4ddvec;
64 q5dd = (pi/180)*q5ddvec;
65
66 qLdd = q5dd;
67
68 %% Denavit-Hartenberg's parameters
69 alpha = [0 pi/2 0 0 pi/2]; % [rad]
```

```

62 a = [0 .150 .250 .220 0]; % [m]
63 d = [.350 0.100 0 0.127 .080]; % [m]
64 theta = [q1; (q2+pi/2); (q3-pi/2); (q4+pi/2); q5]'; % [rad]
65
66 % Payload D-H parameters
67 L = [0 0 .040 0]; % [rad m m rad]
68
69 % WIRE FRAME OF THE ROBOT IN THE INITIAL POSITION
70 % Orientation matrices
71 A10 = denhar_en01(alpha(1),a(1),d(1),theta(1,1));
72 A21 = denhar_en01(alpha(2),a(2),d(2),theta(1,2));
73 A32 = denhar_en01(alpha(3),a(3),d(3),theta(1,3));
74 A43 = denhar_en01(alpha(4),a(4),d(4),theta(1,4));
75 A54 = denhar_en01(alpha(5),a(5),d(5),theta(1,5));
76 AL5 = denhar_en01(L(1),L(2),L(3),L(4));
77
78 A20 = A10*A21;
79 A30 = A20*A32;
80 A40 = A30*A43;
81 A50 = A40*A54;
82 AL0 = A50*AL5;
83
84 % Origins of the reference systems
85 O = [0 0 0];
86 O1 = A10(1:3,4);
87 O2 = A20(1:3,4);
88 O3 = A30(1:3,4);
89 O4 = A40(1:3,4);
90 O5 = A50(1:3,4);
91 OL = AL0(1:3,4);
92
93 % Payload coordinates (rectangle for visualization)
94 aL = [.115, .040, 0, 1]'; % [m]
95 bL = [-.115, .040, 0, 1]'; % Coordinates of the payload frame
96 cL = [-.115, -.040, 0, 1]';
97 dL = [.115, -.040, 0, 1]';
98 aL0 = AL0*aL; % Payload in global reference system
99 bL0 = AL0*bL;
100 cL0 = AL0*cL;
101 dL0 = AL0*dL;
102
103 % Plot wire frame
104 figure()
105 start = plot3([O(1) O1(1) O2(1) O3(1) O4(1) O5(1) OL(1)],...
106 [O(2) O1(2) O2(2) O3(2) O4(2) O5(2) OL(2)],...
107 [O(3) O1(3) O2(3) O3(3) O4(3) O5(3) OL(3)], '-k', 'linewidth',3);
108 hold on
109 axis equal
110 xlabel('axis x')
111 ylabel('axis y')
112 zlabel('axis z')
113 grid on
114
115 % Plot joints
116 R = 0.01; H = 0.05; N = 10;
117 r1 = joint_rev_01(R,H,N,A10,'g'); hold on
118 r2 = joint_rev_01(R,H,N,A20,'g'); hold on
119 r3 = joint_rev_01(R,H,N,A30,'g'); hold on
120 r4 = joint_rev_01(R,H,N,A40,'g'); hold on
121 r5 = joint_rev_01(R,H,N,A50,'r'); hold on
122
123 % Plot base
124 base = fill3([-0.05 .05 .05 -.05],[-.05 -.05 .05 .05],[0 0 0 0],[.5 .5 .5]);
125
126 % Plot payload
127 fill3([aL0(1) bL0(1) cL0(1) dL0(1)], ...
128 [aL0(2) bL0(2) cL0(2) dL0(2)], ...
129 [aL0(3) bL0(3) cL0(3) dL0(3)], [.3 .3 .5])

```



```

127
128         %% WIRE FRAME OF THE ROBOT IN THE FINAL POSITION
129 % Orientation matrices
130 A10f = denhar_en01(alpha(1),a(1),d(1),theta(end,1));
131 A21f = denhar_en01(alpha(2),a(2),d(2),theta(end,2));
132 A32f = denhar_en01(alpha(3),a(3),d(3),theta(end,3));
133 A43f = denhar_en01(alpha(4),a(4),d(4),theta(end,4));
134 A54f = denhar_en01(alpha(5),a(5),d(5),theta(end,5));
135 AL5f = AL5;
136
137 A20f = A10f*A21f;
138 A30f = A20f*A32f;
139 A40f = A30f*A43f;
140 A50f = A40f*A54f;
141 ALOf = A50f*AL5f;
142
143 % Origins of the reference systems
144 O1f = A10f(1:3,4);
145 O2f = A20f(1:3,4);
146 O3f = A30f(1:3,4);
147 O4f = A40f(1:3,4);
148 O5f = A50f(1:3,4);
149 OLf = ALOf(1:3,4);
150
151 % Payload coordinates in final position
152 aLOf = ALOf*aL; % Payload in global reference system
153 bLOf = ALOf*bL;
154 cLOf = ALOf*cL;
155 dLOf = ALOf*dL;
156
157 % Plot wire frame
158 final = plot3([O1f(1) O1f(1) O2f(1) O3f(1) O4f(1) O5f(1) OLf(1)],...
159              [O1f(2) O1f(2) O2f(2) O3f(2) O4f(2) O5f(2) OLf(2)],...
160              [O1f(3) O1f(3) O2f(3) O3f(3) O4f(3) O5f(3) OLf(3)], '-b', 'linewidth'
161              ,2); hold on
162
163 axis equal
164 xlabel('axis x')
165 ylabel('axis y')
166 zlabel('axis z')
167 grid on
168
169 % Plot joints
170 R = 0.01; H = 0.05; N = 10;
171 r1f = joint_rev_01(R,H,N,A10f,'c'); hold on
172 r2f = joint_rev_01(R,H,N,A20f,'c'); hold on
173 r3f = joint_rev_01(R,H,N,A30f,'c'); hold on
174 r4f = joint_rev_01(R,H,N,A40f,'c'); hold on
175 r5f = joint_rev_01(R,H,N,A50f,'r'); hold on
176
177 % Plot payload
178 PL = fill3([aLOf(1) bLOf(1) cLOf(1) dLOf(1)], ...
179           [aLOf(2) bLOf(2) cLOf(2) dLOf(2)], ...
180           [aLOf(3) bLOf(3) cLOf(3) dLOf(3)], [.3 .3 .5]);
181
182 %% Wrist center point CP and End effector trajectory plot
183 % 04 is located on CP
184 % 05 is located on EE
185 CP = ones(4,length(timevec));
186 EE = ones(4,length(timevec));
187
188 for i = 1:length(timevec)
189
190     A10i = denhar_en01(alpha(1),a(1),d(1),theta(i,1));
191     A21i = denhar_en01(alpha(2),a(2),d(2),theta(i,2));
192     A32i = denhar_en01(alpha(3),a(3),d(3),theta(i,3));
193     A43i = denhar_en01(alpha(4),a(4),d(4),theta(i,4));
194     A54i = denhar_en01(alpha(5),a(5),d(5),theta(i,5));

```

```

192
193 A20i = A10i*A21i;
194 A30i = A20i*A32i;
195 A40i = A30i*A43i;
196 A50i = A40i*A54i;
197
198 CP(:,i) = A40i(:,4);
199 EE(:,i) = A50i(:,4);
200
201 end
202
203 CPtrajectory = plot3(CP(1,:),CP(2,:),CP(3:),'y','linewidth',2); hold on
204 EEtrajectory = plot3(EE(1,:),EE(2,:),EE(3:),'m','linewidth',2); hold on
205
206 %% Legend
207 title('Robot wire frame - initial and final positions')
208 set(gcf, 'Color', 'w');
209 legend([start, final, base, PL, CPtrajectory, EEtrajectory], ...
210 'Initial position','Final position','Base','Payload', ...
211 'CP trajectory','EE trajectory')
212
213 %% KINEMATIC (speeds and accelerations) centre of mass of frame 5
214 w0 = [0 0 0]'; wd0 = [0 0 0]';
215 v0 = [0 0 0]'; vd0 = [0 0 0]';
216 vG0 = [0 0 0]'; vdG0 = [0 0 0]';
217 k0 = [0 0 1]'; % equal for each joint in its reference system
218 % lim1 is a position vector taken from iAim1(1:3,4)
219
220 % Revolute or prismatic joint parameter (1/0)
221 delta = zeros(1,6); % revolute joints have delta = 0
222
223 % bi = position vector of center of mass of link i with respect to frame i
224 % expressed in frame i
225 % computed by using exercise's Figure 2
226 b0 = [-0.0525 0 .100]';
227 b1 = [.080 -.02325 -.035]';
228 b2 = [.125 0 0]';
229 b3 = [0 -0.125 0]';
230 b4 = [-.0075 -.0125 .005]';
231 b5 = [0 0 -0.01]';
232 bL = [0 0 .015]';
233
234 % Preallocating vectors' dimension
235 w10vec = ones(3,length(timevec)); % global reference system
236 wd10vec = ones(3,length(timevec));
237 w1vec = ones(3,length(timevec)); % local reference system
238 wd1vec = ones(3,length(timevec));
239 vG1vec = ones(3,length(timevec));
240 vGd1vec = ones(3,length(timevec));
241
242 w20vec = ones(3,length(timevec));
243 wd20vec = ones(3,length(timevec));
244 w2vec = ones(3,length(timevec));
245 wd2vec = ones(3,length(timevec));
246 vG2vec = ones(3,length(timevec));
247 vGd2vec = ones(3,length(timevec));
248
249 w30vec = ones(3,length(timevec));
250 wd30vec = ones(3,length(timevec));
251 w3vec = ones(3,length(timevec));
252 wd3vec = ones(3,length(timevec));
253 vG3vec = ones(3,length(timevec));
254 vGd3vec = ones(3,length(timevec));
255
256 w40vec = ones(3,length(timevec));
257 wd40vec = ones(3,length(timevec));

```

```

257 w4vec = ones(3,length(timevec));
258 wd4vec = ones(3,length(timevec));
259 vG4vec = ones(3,length(timevec));
260 vGd4vec = ones(3,length(timevec));
261
262 w50vec = ones(3,length(timevec));
263 wd50vec = ones(3,length(timevec));
264 w5vec = ones(3,length(timevec));
265 wd5vec = ones(3,length(timevec));
266 vG5vec = ones(3,length(timevec));
267 vGd5vec = ones(3,length(timevec));
268
269 wL0vec = ones(3,length(timevec));
270 wdL0vec = ones(3,length(timevec));
271 wLvec = ones(3,length(timevec));
272 wdLvec = ones(3,length(timevec));
273 vGLvec = ones(3,length(timevec));
274 vGdLvec = ones(3,length(timevec));
275
276 for i = 1:length(timevec)
277
278     A10i = denhar_en01(alpha(1),a(1),d(1),theta(i,1));
279     A01i = inv(A10i);
280
281     A21i = denhar_en01(alpha(2),a(2),d(2),theta(i,2));
282     A12i = inv(A21i);
283
284     A32i = denhar_en01(alpha(3),a(3),d(3),theta(i,3));
285     A23i = inv(A32i);
286
287     A43i = denhar_en01(alpha(4),a(4),d(4),theta(i,4));
288     A34i = inv(A43i);
289
290     A54i = denhar_en01(alpha(5),a(5),d(5),theta(i,5));
291     A45i = inv(A54i);
292
293     AL5 = denhar_en01(L(1),L(2),L(3),L(4));
294     A5L = inv(AL5);
295
296     A20i = A10i*A21i;
297     A30i = A20i*A32i;
298     A40i = A30i*A43i;
299     A50i = A40i*A54i;
300     AL0i = A50i*AL5;
301
302     [w1,wd1,v1,vG1,vd1,vGd1] = kinem_en02(w0,wd0,v0,vd0,k0,A10i(1:3,4),b1,
        delta(1),q1d(i),q1dd(i),A01i(1:3,1:3));
303     w10vec(:,i) = A10i(1:3,1:3)*w1*180/pi;
304     wd10vec(:,i) = A10i(1:3,1:3)*wd1*180/pi;
305     w1vec(:,i) = w1;
306     wd1vec(:,i) = wd1;
307     vG1vec(:,i) = vG1;
308     vGd1vec(:,i) = vGd1;
309
310     [w2,wd2,v2,vG2,vd2,vGd2] = kinem_en02(w1,wd1,v1,vd1,k0,A21i(1:3,4),b2,
        delta(2),q2d(i),q2dd(i),A12i(1:3,1:3));
311     w20vec(:,i) = A20i(1:3,1:3)*w2*180/pi;
312     wd20vec(:,i) = A20i(1:3,1:3)*wd2*180/pi;
313     w2vec(:,i) = w2;
314     wd2vec(:,i) = wd2;
315     vG2vec(:,i) = vG2;
316     vGd2vec(:,i) = vGd2;
317
318     [w3,wd3,v3,vG3,vd3,vGd3] = kinem_en02(w2,wd2,v2,vd2,k0,A32i(1:3,4),b3,
        delta(3),q3d(i),q3dd(i),A23i(1:3,1:3));
319     w30vec(:,i) = A30i(1:3,1:3)*w3*180/pi;

```

```

320 wd30vec(:,i) = A30i(1:3,1:3)*wd3*180/pi;
321 w3vec(:,i) = w3;
322 wd3vec(:,i) = wd3;
323 vG3vec(:,i) = vG3;
324 vGd3vec(:,i) = vGd3;
325
326 [w4,wd4,v4,vG4,vd4,vGd4] = kinem_en02(w3,wd3,v3,vd3,k0,A43i(1:3,4),b4,
    delta(4),q4d(i),q4dd(i),A34i(1:3,1:3));
327 w40vec(:,i) = A40i(1:3,1:3)*w4*180/pi;
328 wd40vec(:,i) = A40i(1:3,1:3)*wd4*180/pi;
329 w4vec(:,i) = w4;
330 wd4vec(:,i) = wd4;
331 vG4vec(:,i) = vG4;
332 vGd4vec(:,i) = vGd4;
333
334 [w5,wd5,v5,vG5,vd5,vGd5] = kinem_en02(w4,wd4,v4,vd4,k0,A54i(1:3,4),b5,
    delta(5),q5d(i),q5dd(i),A45i(1:3,1:3));
335 w5vec(:,i) = w5;
336 wd5vec(:,i) = wd5;
337 w50vec(:,i) = A50i(1:3,1:3)*w5*180/pi;
338 wd50vec(:,i) = A50i(1:3,1:3)*wd5*180/pi;
339 vG5vec(:,i) = vG5;
340 vGd5vec(:,i) = vGd5;
341
342 [wL,wdL,vL,vGL,vdL,vGdL] = kinem_en02(w5,wd5,v5,vd5,k0,A5L(1:3,4),bL,delta
    (6),qLd(i),qLdd(i),A5L(1:3,1:3));
343 wL0vec(:,i) = AL0i(1:3,1:3)*wL*180/pi;
344 wdL0vec(:,i) = AL0i(1:3,1:3)*wdL*180/pi;
345 wLvec(:,i) = wL;
346 wdLvec(:,i) = wdL;
347 vGLvec(:,i) = vGL;
348 vGdLvec(:,i) = vGdL;
349 end
350
351 %% Joint 1 speeds and accelerations
352 figure()
353 subplot(2,2,1)
354 plot(timevec,w10vec)
355 grid on
356 xlabel('Time [s]')
357 ylabel('\omega [deg/s]')
358 title('\omega_1 with respect to global system')
359 set(gcf, 'Color', 'w')
360 legend('\omega_x [deg/s]', '\omega_y [deg/s]', '\omega_z [deg/s]')
361
362 subplot(2,2,2)
363 plot(timevec,wd10vec)
364 grid on
365 xlabel('Time [s]')
366 ylabel('(d\omega/dt) [deg/s^2]')
367 title('\omega_d_1 with respect to global system')
368 set(gcf, 'Color', 'w')
369 legend('\omega_d_x [deg/s^2]', '\omega_d_y [deg/s^2]', '\omega_d_z [deg/s^2]')
370
371 subplot(2,2,3)
372 plot(timevec,vG1vec)
373 grid on
374 xlabel('Time [s]')
375 ylabel('Mass center velocity [m/s]')
376 title('v_G_1')
377 set(gcf, 'Color', 'w')
378 legend('v_G_x [m/s]', 'v_G_y [m/s]', 'v_G_z [m/s]')
379
380 subplot(2,2,4)
381 plot(timevec,vGd1vec)

```

```

382 grid on
383 xlabel('Time [s]')
384 ylabel('Mass center acceleration [m/s^2]')
385 title('a_G_1')
386 set(gcf, 'Color', 'w')
387 legend('a_G_x [m/s^2]', 'a_G_y [m/s^2]', 'a_G_z [m/s^2]')
388
389 %% Joint 2 speeds and accelerations
390 figure()
391 subplot(2,2,1)
392 plot(timevec,w20vec)
393 grid on
394 xlabel('Time [s]')
395 ylabel('\omega [deg/s]')
396 title('\omega_2 with respect to global system')
397 set(gcf, 'Color', 'w')
398 legend('\omega_x [deg/s]', '\omega_y [deg/s]', '\omega_z [deg/s]')
399
400 subplot(2,2,2)
401 plot(timevec,wd20vec)
402 grid on
403 xlabel('Time [s]')
404 ylabel('(d\omega/dt) [deg/s^2]')
405 title('\omega_d_2 with respect to global system')
406 set(gcf, 'Color', 'w')
407 legend('\omega_d_x [deg/s^2]', '\omega_d_y [deg/s^2]', '\omega_d_z [deg/s^2]')
408
409 subplot(2,2,3)
410 plot(timevec,vG2vec)
411 grid on
412 xlabel('Time [s]')
413 ylabel('Mass center velocity [m/s]')
414 title('v_G_2')
415 set(gcf, 'Color', 'w')
416 legend('v_G_x [m/s]', 'v_G_y [m/s]', 'v_G_z [m/s]')
417
418 subplot(2,2,4)
419 plot(timevec,vGd2vec)
420 grid on
421 xlabel('Time [s]')
422 ylabel('Mass center acceleration [m/s^2]')
423 title('a_G_2')
424 set(gcf, 'Color', 'w')
425 legend('a_G_x [m/s^2]', 'a_G_y [m/s^2]', 'a_G_z [m/s^2]')
426
427 %% Joint 3 speeds and accelerations
428 figure()
429 subplot(2,2,1)
430 plot(timevec,w30vec)
431 grid on
432 xlabel('Time [s]')
433 ylabel('\omega [deg/s]')
434 title('\omega_3 with respect to global system')
435 set(gcf, 'Color', 'w')
436 legend('\omega_x [deg/s]', '\omega_y [deg/s]', '\omega_z [deg/s]')
437
438 subplot(2,2,2)
439 plot(timevec,wd30vec)
440 grid on
441 xlabel('Time [s]')
442 ylabel('(d\omega/dt) [deg/s^2]')
443 title('\omega_d_3 with respect to global system')
444 set(gcf, 'Color', 'w')
445 legend('\omega_d_x [deg/s^2]', '\omega_d_y [deg/s^2]', '\omega_d_z [deg/s^2]')

```

```

446
447 subplot(2,2,3)
448 plot(timevec,vG3vec)
449 grid on
450 xlabel('Time [s]')
451 ylabel('Mass center velocity [m/s]')
452 title('v_G_3')
453 set(gcf, 'Color', 'w')
454 legend('v_G_x [m/s]', 'v_G_y [m/s]', 'v_G_z [m/s]')
455
456 subplot(2,2,4)
457 plot(timevec,vGd3vec)
458 grid on
459 xlabel('Time [s]')
460 ylabel('Mass center acceleration [m/s^2]')
461 title('a_G_3')
462 set(gcf, 'Color', 'w')
463 legend('a_G_x [m/s^2]', 'a_G_y [m/s^2]', 'a_G_z [m/s^2]')
464
465 %% Joint 4 speeds and accelerations
466 figure()
467 subplot(2,2,1)
468 plot(timevec,w40vec)
469 grid on
470 xlabel('Time [s]')
471 ylabel('\omega [deg/s]')
472 title('\omega_4 with respect to global system')
473 set(gcf, 'Color', 'w')
474 legend('\omega_x [deg/s]', '\omega_y [deg/s]', '\omega_z [deg/s]')
475
476 subplot(2,2,2)
477 plot(timevec,wd40vec)
478 grid on
479 xlabel('Time [s]')
480 ylabel('(d\omega/dt) [deg/s^2]')
481 title('\omega_d_4 with respect to global system')
482 set(gcf, 'Color', 'w')
483 legend('\omega_d_x [deg/s^2]', '\omega_d_y [deg/s^2]', '\omega_d_z [deg/s^2]')
484
485 subplot(2,2,3)
486 plot(timevec,vG4vec)
487 grid on
488 xlabel('Time [s]')
489 ylabel('Mass center velocity [m/s]')
490 title('v_G_4')
491 set(gcf, 'Color', 'w')
492 legend('v_G_x [m/s]', 'v_G_y [m/s]', 'v_G_z [m/s]')
493
494 subplot(2,2,4)
495 plot(timevec,vGd4vec)
496 grid on
497 xlabel('Time [s]')
498 ylabel('Mass center acceleration [m/s^2]')
499 title('a_G_4')
500 set(gcf, 'Color', 'w')
501 legend('a_G_x [m/s^2]', 'a_G_y [m/s^2]', 'a_G_z [m/s^2]')
502
503 %% Joint 5 speeds and accelerations
504 figure()
505 subplot(1,2,1)
506 plot(timevec,w50vec)
507 grid on
508 xlabel('Time [s]')
509 ylabel('\omega [deg/s]')
510 title('\omega_5 with respect to global system')

```

```

511 set(gcf, 'Color', 'w')
512 legend('\omega_x [deg/s]', '\omega_y [deg/s]', '\omega_z [deg/s]')
513
514 subplot(1,2,2)
515 plot(timevec, wd50vec)
516 grid on
517 xlabel('Time [s]')
518 ylabel('(d\omega/dt) [deg/s^2]')
519 title('\omega_d_5 with respect to global system')
520 set(gcf, 'Color', 'w')
521 legend('\omega_d_x [deg/s^2]', '\omega_d_y [deg/s^2]', '\omega_d_z [deg/s^2]')
522
523 figure()
524 subplot(2,2,1)
525 plot(timevec, w5vec)
526 grid on
527 xlabel('Time [s]')
528 ylabel('\omega [deg/s]')
529 title('\omega_5')
530 set(gcf, 'Color', 'w')
531 legend('\omega_x [deg/s]', '\omega_y [deg/s]', '\omega_z [deg/s]')
532
533 subplot(2,2,2)
534 plot(timevec, wd5vec)
535 grid on
536 xlabel('Time [s]')
537 ylabel('(d\omega/dt) [deg/s^2]')
538 title('\omega_d_5')
539 set(gcf, 'Color', 'w')
540 legend('\omega_d_x [deg/s^2]', '\omega_d_y [deg/s^2]', '\omega_d_z [deg/s^2]')
541
542 subplot(2,2,3)
543 plot(timevec, vG5vec)
544 grid on
545 xlabel('Time [s]')
546 ylabel('Mass center velocity [m/s]')
547 title('v_G_5')
548 set(gcf, 'Color', 'w')
549 legend('v_G_x [m/s]', 'v_G_y [m/s]', 'v_G_z [m/s]')
550
551 subplot(2,2,4)
552 plot(timevec, vGd5vec)
553 grid on
554 xlabel('Time [s]')
555 ylabel('Mass center acceleration [m/s^2]')
556 title('a_G_5')
557 set(gcf, 'Color', 'w')
558 legend('a_G_x [m/s^2]', 'a_G_y [m/s^2]', 'a_G_z [m/s^2]')
559
560 %% INVERSE DYNAMIC
561 % Masses
562 m0=18; m1=10.5; m2=2; m3=6; m4=2; m5=0.5; mL=1; % [kg]
563 % Each frame from 1 to L is a parallelepiped with homogeneous distributed
564 % mass
565 % except for frame 5 which is a cylinder with homogeneous distributed mass
566
567 % Inertial matrices
568 Ixx0 = (1/12)*m0*(0.200^2+0.160^2);
569 Iyy0 = (1/12)*m0*(0.200^2+0.245^2);
570 Izz0 = (1/12)*m0*(0.160^2+0.245^2);
571 IG0 = [Ixx0 0 0; 0 Iyy0 0; 0 0 Izz0];
572
573 Ixx1 = (1/12)*m1*(0.2465^2+0.230^2);
574 Iyy1 = (1/12)*m1*(0.280^2+0.230^2);

```

```

574 Izz1 = (1/12)*m1*(0.2465^2+0.280^2);
575 IG1 = [Ixx1 0 0; 0 Iyy1 0; 0 0 Izz1];
576
577 Ixx2 = (1/12)*m2*(0.080^2+0.0540^2);
578 Iyy2 = (1/12)*m2*(0.0540^2+0.350^2);
579 Izz2 = (1/12)*m2*(0.080^2+0.350^2);
580 IG2 = [Ixx2 0 0; 0 Iyy2 0; 0 0 Izz2];
581
582 Ixx3 = (1/12)*m3*(0.1875^2+0.120^2);
583 Iyy3 = (1/12)*m3*(0.1875^2+0.390^2);
584 Izz3 = (1/12)*m3*(0.390^2+0.120^2);
585 IG3 = [Ixx3 0 0; 0 Iyy3 0; 0 0 Izz3];
586
587 Ixx4=(1/12)*m4*(0.110^2+0.095^2);
588 Iyy4=(1/12)*m4*(0.110^2+0.095^2);
589 Izz4=(1/12)*m4*(0.095^2+0.095^2);
590 IG4=[Ixx4 0 0; 0 Iyy4 0; 0 0 Izz4];
591
592 Ixx5 = m5*((1/16)*0.06^2+(1/12)*0.02^2);
593 Iyy5 = Ixx5;
594 Izz5 = (1/8)*m5*0.06^2;
595 IG5 = [Ixx5 0 0; 0 Iyy5 0; 0 0 Izz5];
596
597 IxxL=(1/12)*mL*(0.030^2+0.080^2);
598 IyyL=(1/12)*mL*(0.030^2+0.230^2);
599 IzzL=(1/12)*mL*(0.080^2+0.230^2);
600 IGL=[IxxL 0 0; 0 IyyL 0; 0 0 IzzL];
601
602 % Preallocating vectors' dimensions
603 FL = ones(3,length(timevec)); ML = ones(3,length(timevec));
604 F5 = ones(3,length(timevec)); M5 = ones(3,length(timevec));
605 F4 = ones(3,length(timevec)); M4 = ones(3,length(timevec));
606 F3 = ones(3,length(timevec)); M3 = ones(3,length(timevec));
607 F2 = ones(3,length(timevec)); M2 = ones(3,length(timevec));
608 F1 = ones(3,length(timevec)); M1 = ones(3,length(timevec));
609 F0 = ones(3,length(timevec)); M0 = ones(3,length(timevec));
610 F0r = ones(length(timevec),1); M0r = ones(length(timevec),1);
611 tau1 = ones(length(timevec),1); tau2 = ones(length(timevec),1);
612 tau3 = ones(length(timevec),1); tau4 = ones(length(timevec),1);
613 tau5 = ones(length(timevec),1);
614
615 for i = 1:length(timevec)
616
617     A10i = denhar_en01(alpha(1),a(1),d(1),theta(i,1));
618     A21i = denhar_en01(alpha(2),a(2),d(2),theta(i,2));
619     A32i = denhar_en01(alpha(3),a(3),d(3),theta(i,3));
620     A43i = denhar_en01(alpha(4),a(4),d(4),theta(i,4));
621     A54i = denhar_en01(alpha(5),a(5),d(5),theta(i,5));
622
623     AL5i = denhar_en01(L(1),L(2),L(3),L(4));
624
625     A20i = A10i*A21i;
626     A30i = A20i*A32i;
627     A40i = A30i*A43i;
628     A50i = A40i*A54i;
629     AL0i = A50i*AL5i;
630
631     [FL(:,i),ML(:,i)] = dynam_en02([0 0 0]', [0 0 0]', mL, vGdLvec(:,i), wLvec
        (:,i), wdLvec(:,i), [0 0 0]', bL, IGL, eye(3), inv(AL0i(1:3,1:3)));
632     [F5(:,i),M5(:,i)] = dynam_en02(FL(:,i), ML(:,i), m5, vGd5vec(:,i), w5vec(:,
        i), wd5vec(:,i),AL5(1:3,4), b5, IG5, AL5i(1:3,1:3), inv(A50i(1:3,1:3)))
        ;
633     [F4(:,i),M4(:,i)] = dynam_en02(F5(:,i), M5(:,i), m4, vGd4vec(:,i), w4vec(:,
        i), wd4vec(:,i),A54(1:3,4), b4, IG4, A54i(1:3,1:3), inv(A40i(1:3,1:3)))
        ;

```



```

634 [F3(:,i),M3(:,i)] = dynam_en02(F4(:,i), M4(:,i), m3, vGd3vec(:,i), w3vec(:,
    i), wd3vec(:,i),A43(1:3,4), b3, IG3, A43i(1:3,1:3), inv(A30i(1:3,1:3)))
    ;
635 [F2(:,i),M2(:,i)] = dynam_en02(F3(:,i), M3(:,i), m2, vGd2vec(:,i), w2vec(:,
    i), wd2vec(:,i),A32(1:3,4), b2, IG2, A32i(1:3,1:3), inv(A20i(1:3,1:3)))
    ;
636 [F1(:,i),M1(:,i)] = dynam_en02(F2(:,i), M2(:,i), m1, vGd1vec(:,i), w1vec(:,
    i), wd1vec(:,i),A21(1:3,4), b1, IG1, A21i(1:3,1:3), inv(A10i(1:3,1:3)))
    ;
637 [F0(:,i),M0(:,i)] = dynam_en02(F1(:,i), M1(:,i), m0, vdG0 , w0 , wd0 ,A10
    (1:3,4), b0, IG0, A10i(1:3,1:3), eye(3));
638
639 % Resultants forces and torques exerted by base 0
640 F0r(i) = sqrt(F0(1,i)^2+F0(2,i)^2+F0(3,i)^2);
641 M0r(i) = sqrt(M0(1,i)^2+M0(2,i)^2+M0(3,i)^2);
642
643 % Actuator torques
644 tau1(i) = delta(1)*k0'*F1(:,i)+(1-delta(1))*k0'*M1(:,i);
645 tau2(i) = delta(2)*k0'*F2(:,i)+(1-delta(2))*k0'*M2(:,i);
646 tau3(i) = delta(3)*k0'*F3(:,i)+(1-delta(3))*k0'*M3(:,i);
647 tau4(i) = delta(4)*k0'*F4(:,i)+(1-delta(4))*k0'*M4(:,i);
648 tau5(i) = delta(5)*k0'*F5(:,i)+(1-delta(5))*k0'*M5(:,i);
649 end
650
651 % Forces exerted by base 0 to the floor
652 F0toF1 = -F0;
653 M0toF1 = -M0;
654
655
656
657 %% PLOT ACTUATOR TORQUES
658 figure()
659 plot(timevec,tau1(:,1),timevec,tau2(:,1),timevec,tau3(:,1),timevec,tau4
    (:,1),timevec,tau5(:,1),'linewidth',1.5)
660 title('ACTUATOR TORQUES')
661 xlabel('Time [s]')
662 ylabel('Torque [Nm]')
663 legend('Tau 1','Tau 2','Tau 3','Tau 4','Tau 5')
664 set(gcf, 'Color', 'w')
665 grid on
666
667 %% PLOT FORCES AND TORQUES EXERTED BY BASE 0 TO FLOOR
668 figure()
669 subplot(1,2,1)
670 plot(timevec,F0toF1(1,:),timevec,F0toF1(2,:),timevec,F0toF1(3,:),timevec,
    F0r(:,1),'linewidth',1.5)
671 title('FORCES TRANSMITTED TO FLOOR')
672 xlabel('Time [s]')
673 ylabel('Force [N]')
674 legend('X component','Y component','Z component','Resultant')
675 set(gcf, 'Color', 'w')
676 grid on
677
678 subplot(1,2,2)
679 plot(timevec,M0toF1(1,:),timevec,M0toF1(2,:),timevec,M0toF1(3,:),timevec,
    M0r(:,1),'linewidth',1.5)
680 title('TORQUES TRANSMITTED TO FLOOR')
681 xlabel('Time [s]')
682 ylabel('Torque [Nm]')
683 legend('X component','Y component','Z component','Resultant')
684 set(gcf, 'Color', 'w')
685 grid on
686
687 function A=denhar_en01(alfa,a,d,teta)
688
689

```

```

690
691 A=[cos(teta)          -sin(teta)          0          a
692     sin(teta)*cos(alfa) cos(teta)*cos(alfa) -sin(alfa) -d*sin(alfa)
693     sin(teta)*sin(alfa) cos(teta)*sin(alfa) cos(alfa)  d*cos(alfa)
694     0                  0                  0          1];
695
696 end
697
698 function [h]=joint_rev_01(R,H,N,A,col)
699
700
701 [xc,yc,zc1] = cylinder(R, N);
702 [h] = [xc,yc,zc1];
703 zc=zc1*H-H/2;
704 base1=A*[xc(1,:); yc(1,:);zc(1,:);ones(1,N+1)];
705 base2=A*[xc(2,:); yc(2,:);zc(2,:);ones(1,N+1)];
706 xc0=[base1(1,:);base2(1,:)];
707 yc0=[base1(2,:);base2(2,:)];
708 zc0=[base1(3,:);base2(3,:)];
709 surface(xc0,yc0,zc0,'FaceColor', col)
710
711 end
712
713 function [wi,wip,vi,vGi,vip,vGip]=kinem_en02(wim1,wimlp,vim1,vimlp,ki,lim1,
714     bi,deltai,qip,qipp,iAim1)
715
716 %computation of angular velocity omega(i)
717 wi = iAim1*wim1+qip*(1-deltai)*ki;
718
719 %computation of linear velocity v(i) of origin Oi in frame i
720 vi=iAim1*vim1+iAim1*cross(wim1,lim1)+qip*deltai*ki;
721
722 %computation of linear velocity vG(i) of centre of mass in frame i
723 vGi=vi+cross(wi,bi);
724
725 %computation of angular acceleration omegadot(i)
726 wip = iAim1*wimlp+qipp*(1-deltai)*ki+qip*(1-deltai)*cross(iAim1*wim1,ki);
727
728 %computation of linear acceleration vdot(i)
729 vip = iAim1*(vimlp+cross(wimlp,lim1)+cross(wim1,cross(wim1,lim1)))+qipp*
730     deltai*ki+2*qip*deltai*cross(iAim1*wim1,ki);
731
732 %computation of linear acceleration vGdot(i)
733 vGip = vip+cross(wip,bi)+cross(wi,cross(wi,bi));
734
735 end
736
737 function [Fi,Mi]=dynam_en02(Fip1, Mip1, mi, vGip, wi, wip, li, bi, Ii,
738     iAip1, iA0)
739
740 %computation of force Fi
741 Fi = iAip1*Fip1+mi*vGip-mi*iA0*[0 0 -9.81]';
742
743 %computation of moment Mi
744 Mi = iAip1*Mip1-cross(Fi,bi)-cross(iAip1*Fip1,(li-bi))+Ii*wip+cross(wi,Ii*
745     wi);
746 end

```